Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026

Contents

Part I.
Foundations

1. What is Deep Learning?

1.1. Demonstration

▹ Session 1, Slides 1-12

Before any theory, consider the following experiment. A large language model - specifically Claude - is prompted with a single sentence:

> *"Write me a PyTorch script that trains a neural network to classify CIFAR-10 images into 10 classes. Use a simple MLP architecture. Print the training loss each epoch and plot a few example predictions with their true and predicted labels at the end."*

The model produces a complete, runnable Python script. When executed, the script downloads the CIFAR-10 dataset - 60,000 colour images of 32×32 pixels across 10 classes - trains a neural network, and achieves reasonable classification accuracy. A second prompt requesting a CNN instead of an MLP produces a different script that achieves noticeably higher accuracy on the same task.

This demonstration works. It is genuinely easy. And therein lies the problem: *what just happened?*

The script contains weights, layers, a loss function, a training loop, gradient updates. None of these were explained. The goal of this course is to open that black box - not to prompt better, but to think deeper.

1.2. What is Deep Learning?

▹ Session 1: What is Deep Learning?

Artificial intelligence, machine learning, and deep learning are often used interchangeably in popular discourse, but they refer to distinct and nested concepts.

Artificial intelligence is the broadest term - it encompasses any technique that enables machines to exhibit behaviour we would consider intelligent, including rule-based expert systems, search algorithms, and learned models.

Machine learning is a subfield of AI in which systems learn from data rather than being programmed with explicit rules. Instead of writing down every decision, we provide examples and let the system discover the underlying patterns.

Deep learning is a subfield of machine learning that uses neural networks with many layers - so-called deep neural networks. The depth allows the network to learn increasingly abstract representations of the input data.

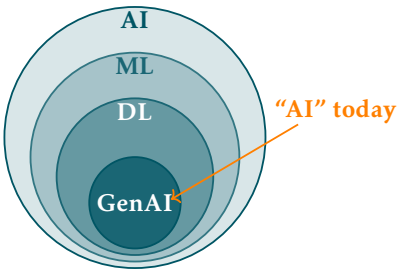

Figure 1.1: The relationship between AI, machine learning, deep learning, and generative AI.

Generative AI - encompassing large language models as well as image, video, and music generation systems - is a specific and currently prominent subset of deep learning. What the general public refers to as "AI" today is almost exclusively generative AI, which sits at the innermost level of this hierarchy.

Definition 1 (Deep Learning)**.** Deep learning is a class of machine learning methods that use multi-layered neural networks to learn hierarchical representations directly from raw data.

This definition will sharpen as the course progresses. For now it is sufficient to hold it loosely - by the end of the course every word in it will have a concrete meaning.

1.3. Brief History

▹ Session 1: A Brief History of Deep Learning

Figure 1.2 shows the key milestones in the development of deep learning alongside the periods of reduced funding and interest known as AI winters. For a comprehensive overview of the field's history see LeCun, Bengio, and G. Hinton 2015 and Goodfellow, Bengio, and Courville 2016.

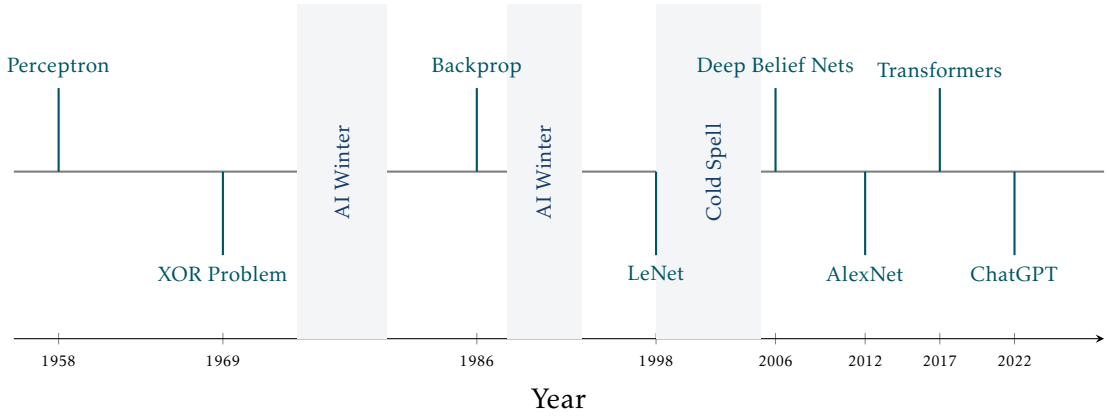

Figure 1.2: Key milestones in the history of deep learning. Shaded regions indicate AI winters and the cold spell of the early 2000s.

1958 - Perceptron. Frank Rosenblatt 1958 introduced the perceptron - the first trainable neural network. It could learn to classify linearly separable patterns by adjusting weighted connections between inputs and a single output unit. The result generated enormous excitement and equally enormous overpromising about the future of machine intelligence. Rosenblatt himself claimed the perceptron would eventually be able to walk, talk, see, write, reproduce itself, and be conscious of its existence - a level of optimism that would later contribute to the severity of the backlash.

The years following the perceptron saw genuine progress. The ADALINE model Widrow and Hoff 1960 introduced the least mean squares learning rule, which remains in use today. The concept of multilayer networks was understood theoretically, and researchers were actively exploring how to train them. There was a widespread belief that human-level artificial intelligence was only a decade away - a belief shared by many prominent scientists and echoed in generous research funding from DARPA and other agencies.

However, the limitations of single-layer networks were becoming apparent in practice. Many of the problems researchers wanted to solve - pattern recognition, language understanding, game playing - turned out to be far harder than anticipated. The gap between the promise of the perceptron and the reality of what it could achieve was widening.

1969 - XOR Problem. Minsky and Papert 1969 published *Perceptrons*, demonstrating formally that single-layer networks could not solve problems as simple as the XOR

function. The 1970s were a difficult period for neural network research. The Lighthill Report Lighthill 1973 delivered a damning assessment of AI research in 1973, concluding that the field had failed to deliver on its promises. DARPA cut funding significantly and many researchers moved away from neural networks entirely. This was the first AI winter.

Yet work continued quietly. Paul Werbos derived the backpropagation algorithm in his 1974 PhD thesis Werbos 1974 - years before it became widely known. Fukushima 1980 introduced the Neocognitron, a hierarchical neural network inspired by the visual cortex that anticipated many ideas later formalised in convolutional networks. Hopfield 1982 networks demonstrated that recurrent networks could function as associative memories. These contributions kept the field alive through the cold years, largely unnoticed by the mainstream AI community which had shifted its focus toward symbolic reasoning and expert systems.

Japan's role in this period deserves particular mention. Fukushima's Neocognitron emerged from NHK Science and Technology Research Laboratories in Tokyo. In 1982 the Japanese government launched the ambitious Fifth Generation Computer Project Ministry of International Trade and Industry 1982, investing heavily in AI hardware and parallel computing. The project caused significant alarm in the US and UK and triggered substantial counter-investment in AI research on both sides of the Atlantic. Its ultimate failure to deliver human-level reasoning contributed to the disillusionment of the second AI winter.

1986 - Backpropagation. Rumelhart, G. E. Hinton, and Williams 1986 popularised the backpropagation algorithm, providing a practical method for training multi-layer networks by propagating error gradients backwards through the layers. The revival of backpropagation generated enormous enthusiasm. Multilayer networks were successfully applied to speech recognition, image classification, and financial forecasting. LeCun applied backpropagation to convolutional networks at Bell Labs, demonstrating practical handwriting recognition systems used by the US postal service. Hochreiter and Schmidhuber introduced the Long Short-Term Memory (LSTM) network in 1997 Hochreiter and Schmidhuber 1997b, solving the vanishing gradient problem for recurrent networks and laying the foundation for sequence modelling over the following two decades.

However, the second AI winter arrived in the late 1980s and early 1990s. Expert systems - rule-based AI programs that had attracted enormous commercial investment - collapsed as their maintenance costs proved prohibitive and their brittleness became apparent. The Lisp machine market crashed in 1987. DARPA again cut funding. Neural networks hit their own practical ceiling: training deep networks remained unstable, datasets were small, and support vector machines (SVMs) - introduced by Vapnik 1995 - offered better performance on many tasks with stronger theoretical guarantees. By the mid-1990s SVMs had largely displaced neural networks as the method of choice for classification problems.

1998 - LeNet. Yann LeCun, Bottou, et al. 1998 demonstrated convolutional neural networks for handwritten digit recognition, achieving state-of-the-art results on the MNIST dataset. Despite its success, neural networks remained out of favour through the early 2000s - a period sometimes called the cold spell - as support vector machines and other kernel methods dominated the field.

This cold spell was not a complete winter - small groups continued working on deep architectures. Bengio, Delalleau, and Le Roux 2004 explored the difficulty of training

deep networks, identifying the vanishing gradient problem as the central obstacle. Hinton's group at Toronto worked on unsupervised pretraining as a way to initialise deep networks before fine-tuning with backpropagation. The theoretical groundwork for the 2006 breakthrough was being laid quietly throughout this period.

2006 - Deep Belief Nets. G. E. Hinton, Osindero, and Teh 2006 showed that deep networks could be trained effectively using a layer-wise pretraining procedure . This reignited interest in deep architectures and marked the beginning of the modern deep learning era, even before large datasets and GPU training were fully established.

The key developments of this period were infrastructural rather than algorithmic. Raina, Madhavan, and Ng 2009 demonstrated that GPUs could accelerate neural network training by an order of magnitude, making experiments that previously took weeks feasible in hours. The ImageNet dataset was released by Deng et al. 2009, providing for the first time a benchmark large and challenging enough to reveal the true potential of deep architectures. Nair and G. E. Hinton 2010 introduced the ReLU activation function, which proved dramatically more effective than sigmoid and tanh activations for training deep networks. These three developments - GPU training, large-scale data, and better activations - set the stage for AlexNet's breakthrough in 2012.

2012 - AlexNet. Krizhevsky, Sutskever, and G. E. Hinton 2012 entered a deep convolutional network trained on GPUs into the ImageNet challenge and won by a margin that shocked the field - reducing the top-5 error rate from 26% to 15% in a single year. This moment is widely regarded as the turning point that established deep learning as the dominant paradigm in machine learning.

2017 - Transformers. Vaswani et al. 2017 introduced the transformer architecture in the paper *Attention Is All You Need*, replacing recurrent computation with self-attention mechanisms. The transformer became the foundation of virtually all modern large-scale models in both language and vision.

2022 - ChatGPT. OpenAI released ChatGPT, bringing large language models to a mass audience for the first time. Within two months it had reached 100 million users, making it the fastest-growing consumer application in history and triggering a wave of public and institutional interest in artificial intelligence that continues today.

The pattern visible in Figure 1.2 is worth noting: the history of deep learning is one of recurring cycles of excitement and disappointment, punctuated by genuine breakthroughs. The current era is distinguished not just by the scale of the results but by the breadth of their impact - for the first time, deep learning systems are changing how ordinary people work and communicate on a daily basis.

1.4. Why Now?

▹ Session 1: Why Now? - Four Driving Forces

Deep learning is not a new idea - neural networks have existed since the 1950s. What changed is the confluence of four factors that together made the approach viable at scale. None of these alone was sufficient; together they triggered a revolution.

1.4.1. Data

▹ Session 1: Why Now? - Four Driving Forces, Data - Scale Revolution

The internet created the largest dataset in history by accident. Every uploaded image, every written caption, every search query became potential training material. This was crystallised deliberately in benchmark datasets - ImageNet (2009) provided 1.2 million labelled images across 1,000 categories, and GLUE provided a standardised benchmark

for natural language understanding. These benchmarks did not just measure progress, they created it by giving the community a shared target to optimise against.

Crucially, the field also learned that labels are not always necessary. Self-supervised and weakly supervised learning techniques extract signal directly from unlabelled data, massively expanding what counts as usable training material. The web, in its entirety, became a dataset.

Figure 1.3 shows the growth in dataset scale over time. Two aspects deserve attention. First, the left axis is logarithmic - each gridline represents a tenfold increase, so the jump from ImageNet to LAION-5B represents roughly four thousand times more data. Second, the orange line shows that resolution grew alongside quantity - from 28×28 pixels for MNIST to 1024×1024 for LAION-5B, a 1,000-fold increase in pixels per image. The two dimensions of scale - how many images and how much information per image - grew together.

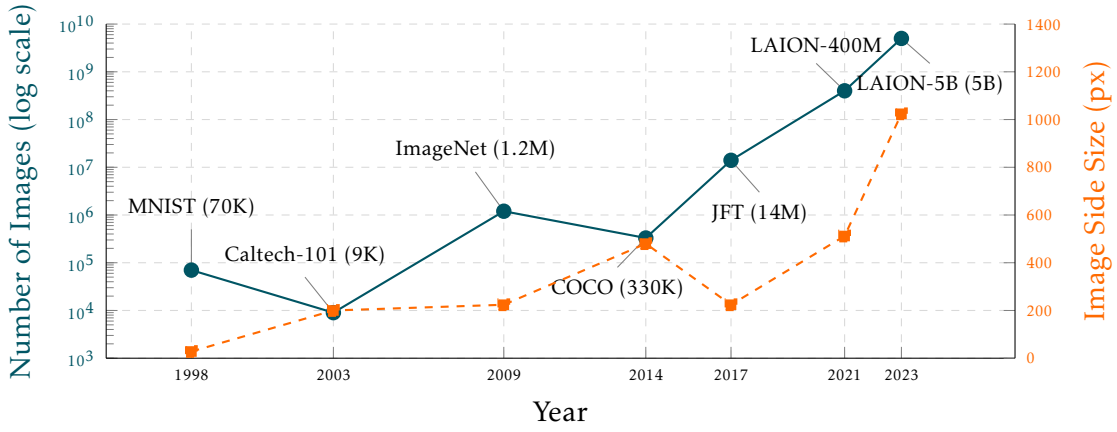

Figure 1.3: Image dataset sizes and resolutions over time. Blue line: number of images (log scale, left axis). Orange line: image side size in pixels (linear scale, right axis).

1.4.2. Compute

▹ Session 1: Compute & Progress

Graphics processing units, designed for the parallel computation of gaming graphics, turned out to be ideally suited for the matrix multiplications at the heart of neural network training. NVIDIA's CUDA programming model made GPUs accessible to researchers without hardware expertise. Custom chips - Google's TPUs, Apple Silicon, and a wave of AI-specific inference hardware - followed, with architectures designed around deep learning from the ground up.

Cloud platforms such as AWS, Microsoft Azure, and Google Cloud democratised access to large-scale compute - a researcher can now rent state-of-the-art hardware by the hour. Learning platforms such as Google Colab, Kaggle Notebooks, and Lightning.ai went further, providing free GPU access directly in the browser with zero setup. As Figure 1.4 shows, GPU performance has grown from single-digit TFLOPS in 2012 to nearly 2,000 TFLOPS in 2024.

1.4.3. Algorithms

▹ Session 1: Why Now? - Four Driving Forces, Compute & Progress

Several targeted innovations removed practical obstacles that had blocked progress for decades. At the architectural level, the multilayer perceptron (MLP), convolutional neu-

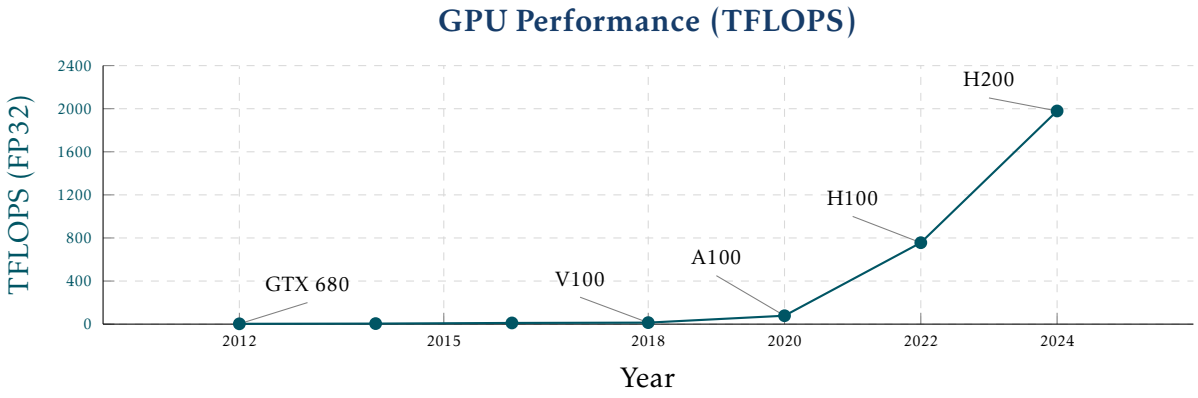

Figure 1.4: GPU performance in TFLOPS (FP32) from 2012 to 2024.

ral network (CNN), and recurrent neural network (RNN) established the foundational building blocks, all trained via the backpropagation algorithm. The ReLU activation function mitigated the vanishing gradient problem that had plagued deep networks. Batch normalisation stabilised training. Dropout provided effective regularisation.

At a higher level, residual connections (ResNets) made it possible to train networks of hundreds of layers. The attention mechanism and the transformer architecture, introduced in 2017, replaced recurrent computation entirely and became the foundation of modern language and vision models.

Figure 1.5 illustrates the impact of these algorithmic advances on the ImageNet benchmark. In 2011, the best classical methods using SVMs achieved around 74% top-5 accuracy. AlexNet's jump to 84.7% in 2012 marked the beginning of the deep learning era. Each subsequent architectural innovation - VGG, ResNet, EfficientNet, ViT - pushed accuracy further, surpassing human performance of approximately 95% around 2015 and reaching over 99% by 2022.

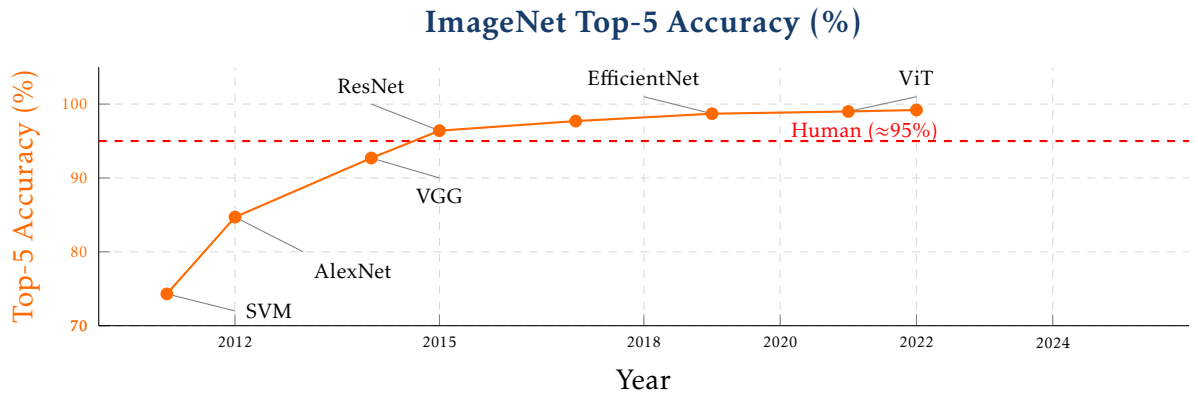

Figure 1.5: ImageNet top-5 accuracy from 2011 to 2022. Each labelled point corresponds to a key architectural innovation. The dashed red line indicates approximate human performance.

1.4.4. Community

Perhaps the most underappreciated factor is the open culture of the deep learning community. Frameworks such as PyTorch, TensorFlow, and JAX are open source,

actively maintained, and freely available. The norm of releasing code alongside papers - through platforms such as Papers with Code, Kaggle, and HuggingFace - means that results are reproducible and immediately buildable upon. Research is shared on arXiv before peer review, and major conferences such as NeurIPS, ICML, and ICLR publish proceedings openly.

This compounding effect - where each breakthrough immediately becomes everyone's foundation - has no parallel in most other scientific fields.

1.5. What Can Deep Learning Do?

▹ Session 1: What Can Deep Learning Do?

Deep learning today touches almost every domain of human activity. Rather than organising applications by the models used, it is more instructive to group them by the type of task they perform.

1.5.1. Language and Communication

Natural language was one of the earliest and remains one of the most impactful application areas. Machine translation systems such as DeepL and Google Translate now produce fluent output across dozens of language pairs, a task that resisted decades of rule-based approaches. Conversational agents - ChatGPT, Claude, Gemini - have brought language models to a mass audience, demonstrating capabilities in reasoning, summarisation, and question answering that were considered out of reach just a few years ago. Code generation tools such as GitHub Copilot and Cursor assist developers by completing, explaining, and generating code in real time.

1.5.2. Vision and Perception

Computer vision was the domain in which deep learning first demonstrated decisive superiority over classical methods, with AlexNet's 2012 ImageNet result marking the turning point. Today deep learning powers image classification, object detection, and semantic segmentation across a vast range of applications. Autonomous driving systems parse complex road scenes in real time, combining camera, lidar, and radar inputs to make driving decisions. Satellite and aerial imagery analysis enables large-scale monitoring of land use, deforestation, and infrastructure.

1.5.3. Healthcare and Medicine

Medical imaging has been transformed by deep learning. Systems trained on large collections of labelled scans can detect tumours, classify X-rays, and segment anatomical structures with accuracy approaching or exceeding that of specialist clinicians. Beyond imaging, deep learning is applied to genomics - identifying disease-associated variants in DNA sequences - and to clinical decision support systems that assist physicians with diagnosis and treatment planning.

1.5.4. Generation

Generative models have emerged as one of the most publicly visible applications of deep learning. Image generation systems such as Stable Diffusion and DALL-E produce photorealistic or stylised images from text descriptions. Video and music

generation - tools such as Suno and Udio - extend this capability to temporal media. The same underlying technology that enables generation also enables detection: deepfake detection systems attempt to identify synthetically generated media, an increasingly important capability as generation quality improves.

1.5.5. Scientific Discovery

Perhaps the most consequential application of deep learning to date is AlphaFold Jumper, Evans, Pritzel, et al. 2021, DeepMind's system for predicting protein three-dimensional structure from amino acid sequence. AlphaFold solved a fifty-year-old open problem in structural biology and earned Demis Hassabis and John Jumper the 2024 Nobel Prize in Chemistry - the first Nobel Prize awarded for work driven by deep learning. Deep learning is also applied to drug discovery - predicting molecular properties and identifying candidate compounds - and to climate and weather modelling, where neural networks are beginning to challenge traditional physics-based simulation approaches.

1.5.6. Decision Making and Control

Reinforcement learning - a branch of deep learning in which agents learn by interacting with an environment - has produced some of the field's most dramatic results. AlphaGo Silver, Huang, Maddison, et al. 2016 defeated the world champion Go player in 2016, a milestone considered a decade away by most experts at the time. Robotic systems learn manipulation and locomotion skills directly from experience, without hand-coded controllers. Recommendation systems - the algorithms that determine what appears in TikTok feeds, Netflix homepages, and Spotify playlists - are among the most economically impactful applications of deep learning, shaping the daily media consumption of billions of people.

2. Shallow Neural Networks

▹ Session 2

These notes accompany the second lecture of the course. The goal of this session is to build up, from first principles, the simplest possible neural network - a so-called **shallow** or **one-hidden-layer** neural network - and to understand precisely what it can represent. No optimisation is discussed here; that comes in later sessions. The question we ask is purely one of expressiveness: what class of functions can such a network compute?

2.1. Supervised Learning

▹ Session 2: Supervised Learning

We begin with the general framework that motivates everything that follows. In **supervised learning** we are given a dataset of input-output pairs

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N} \tag{2.1}$$

where each input $\mathbf{x}^{(i)} \in \mathcal{X}$ belongs to some input space $\mathcal{X}$ and each output $y^{(i)} \in \mathcal{Y}$ belongs to some output space $\mathcal{Y}$. We assume that there exists a true underlying relationship - a mapping $f^* : \mathcal{X} \to \mathcal{Y}$ such that $y = f^*(\mathbf{x})$, but we never have direct access to f^*. We only have access to the finitely many observations in the dataset $\mathcal{D}$.

The goal is to learn a parameterised function $f_\theta : \mathcal{X} \to \mathcal{Y}$ from $\mathcal{D}$ such that it approximates the true unkonwn mapping $f_\theta \approx f^*$. The notation $f_\theta(\mathbf{x}) = f(\mathbf{x};\theta) = \hat{y}$ will be used throughout the course: $\hat{y}$ denotes the model's prediction for input $\mathbf{x}$ given parameters θ. The goal is not merely to fit the training data, but to predict accurately on ***unseen*** inputs - this is the **generalisation** requirement, which we will revisit in depth in later sessions.

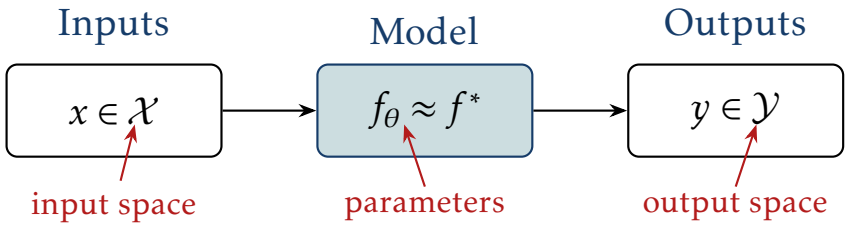

Figure 2.1: Supervised learning: given data pairs $(\mathbf{x}, y)$, learn a model f_θ that approximates the true relationship f^*.

Supervised learning encompasses two fundamental task types. In **regression**, the output space is continuous: $\mathcal{Y} = \mathbb{R}$ (or more generally $\mathbb{R}^k$), and the goal is to predict a numerical value. In **classification**, the output space is discrete, and the goal is to assign each input to one of two categories. These two tasks are illustrated in Figure 2.2 and Figure 2.3.

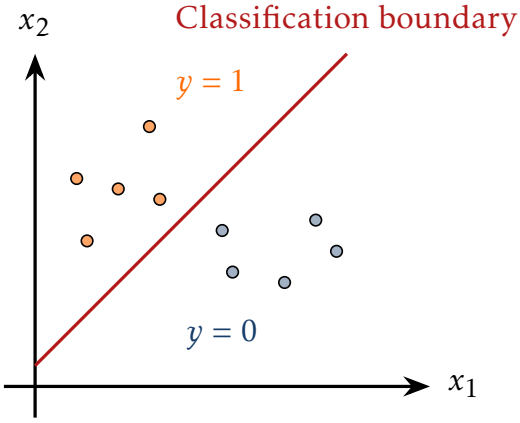

Figure 2.2: Binary classification: two classes separated by a decision boundary.

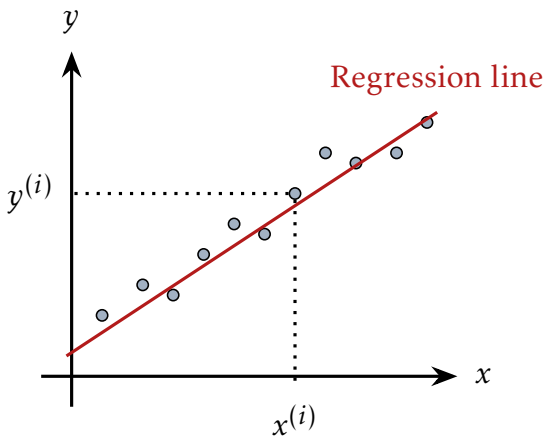

Figure 2.3: Regression: fitting a continuous function to data points.

2.2. Regression and the Linear Model

▸ Session 2: Regression and Linear Model

We focus first on the regression setting with a single input $x \in \mathbb{R}$ and a single output $y \in \mathbb{R}$.

The simplest parameterised function we will work with is an affine function - often called simply **linear model** in the machine learning literature:

$$\hat{y} = f_\theta(x) = \theta_1 x + \theta_0, \qquad \boldsymbol{\theta} = (\theta_0, \theta_1) \in \mathbb{R}^2. \tag{2.2}$$

This model has two parameters: a slope θ_1 and an intercept θ_0 and can be represented graphically as a single **neuron**, shown in Figure 2.4. The input x and the constant 1 (carrying the bias θ_0) feed into the neuron, which computes the weighted sum and produces the output $\hat{y}$. This graphical representation may seem unnecessary for such a simple model, but it will prove invaluable as we build more complex networks - the same diagram extends naturally to multiple inputs, multiple neurons, and multiple layers.

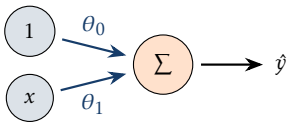

Figure 2.4: Graphical representation of the linear model as a single neuron.

The representational capacity of the linear model is severely limited - it can only fit data that lies approximately along a straight line. When the true relationship f^* is nonlinear, a linear model will fail systematically, as illustrated in Figure 2.6: it will underfit the data regardless of how the parameters are chosen. This is not a failure of optimisation; it is a failure of expressiveness. The model class is simply too small to contain the true function.

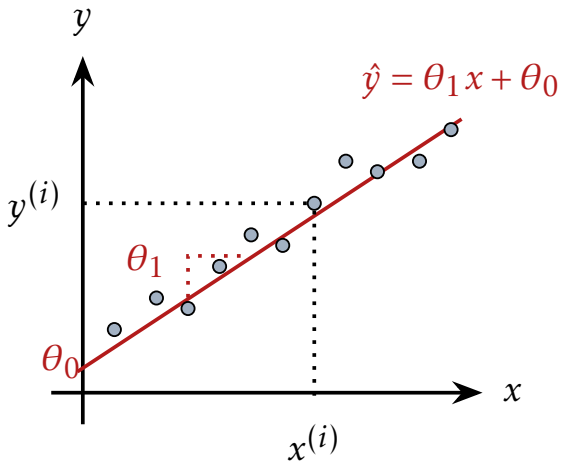

Figure 2.5: A linear model fits the data well when the true relationship is linear.

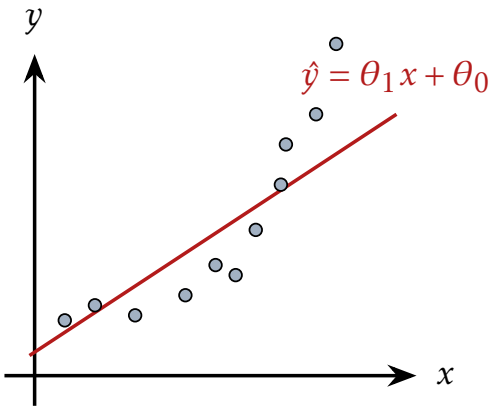

Figure 2.6: A linear model fails to capture nonlinear relationships in the data.

This observation motivates the central question of the lecture: how can we construct a richer class of functions f_θ that can capture nonlinear relationships?

2.3. Nonlinear Regression and Activation Functions

▸ Session 2: Nonlinear Regression

The linear model introduced in the previous section has a fundamental limitation: it can only represent affine relationships between inputs and outputs. To move beyond this, we introduce a **nonlinear activation function** $a : \mathbb{R} \to \mathbb{R}$ applied to the output of the linear computation. The neuron now computes:

$$\hat{y} = f_\theta(x) = a(\theta_1 x + \theta_0). \tag{2.3}$$

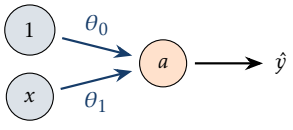

Figure 2.7: A nonlinear neuron applies activation function a to the linear combination of its inputs.

The activation function is what gives the neural network its expressive power -changing the mapping between inputs $\mathcal{X}$ and outputs $\mathcal{Y}$ from linear to nonlinear. The graphical representation of this nonlinear neuron is shown in

Figure 2.7: it is identical to the linear neuron, but the node now carries the symbol a to indicate that a nonlinearity is applied before the output is produced.

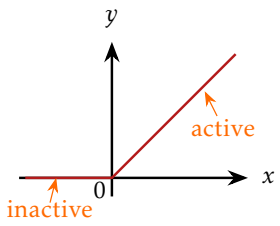

Figure 2.8: The ReLU activation function: inactive (zero) for $x \leq 0$, active (linear) for $x > 0$.

The choice of activation function matters. The most widely used activation function in modern deep learning is the **Rectified Linear Unit**, or **ReLU** (Nair and G. E. Hinton 2010; Glorot, Bordes, and Bengio 2011):

$$a(x) = \text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \quad \text{(active)} \\ 0 & x \leq 0 \quad \text{(inactive).} \end{cases} \tag{2.4}$$

The ReLU is remarkably simple: it passes positive values unchanged and clamps negative values to zero. A **ReLU neuron** is said to be **active** when its pre-activation value is positive, and **inactive** otherwise. Despite its simplicity - or perhaps because of it - the ReLU is extremely effective in practice and will be our default activation function throughout this course. We return to a broader comparison of activation functions in a later session.

Composing the ReLU with a linear function introduces exactly one breakpoint, at $x = -\theta_0/\theta_1$, where the output transitions from inactive (zero) to active (rising linearly). This single bend is still not expressive enough to capture complex nonlinear relationships. The key insight, developed in the next section, is that a *weighted sum* of such ReLU activated functions produces a piecewise linear function with one bend per ReLU - and piecewise linear functions can approximate any continuous function.

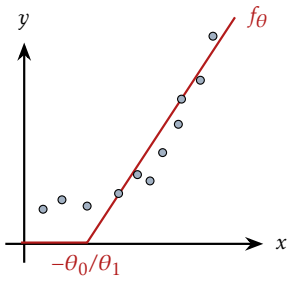

Figure 2.9: A single ReLU applied to a linear function produces one breakpoint at $x = -\theta_0/\theta_1$.

2.4. Combining ReLUs: Piecewise Linear Functions

▸ Session 2: Combining Multiple ReLUs

A single ReLU neuron introduces one breakpoint and is therefore only marginally more expressive than a linear model. The power comes from combining multiple such neurons. Consider a weighted sum of k ReLU-activated linear functions:

$$\hat{y} = f_\theta(x) = \theta_0 + \sum_{j=1}^{k} \theta_j \, a(\theta_{j1} x + \theta_{j0}). \tag{2.5}$$

Each term contributes one breakpoint at $x = -\theta_{j0}/\theta_{j1}$. The resulting function is **piecewise linear** - linear within each interval between breakpoints, with up to k bends in total. As k increases, the function can capture increasingly complex shapes, as illustrated in Figure 2.10.

The graphical representation of this computation for $k = 2$ ReLU activations is shown in Figure 2.11. Each **hidden neuron** j - so called because its activations are internal to the network and not directly observed as inputs or outputs - receives the input x and independently computes $h_j = a(\theta_{j1} x + \theta_{j0})$. Note that in the full representation (left), the input x and bias constant 1 are shown separately for each neuron — this makes explicit that each neuron has its own independent set of parameters $(\theta_{j1}, \theta_{j0})$, even

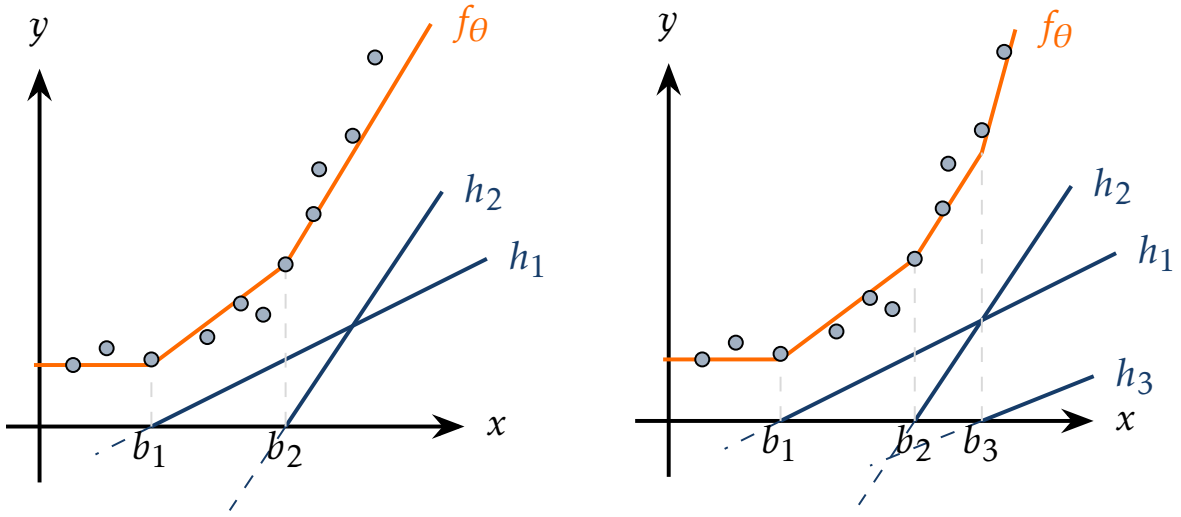

Figure 2.10: Left: combination of two ReLU functions producing two breakpoint. Right: combination of three ReLU functions producing three breakpoints. More neurons yield more expressive functions.

though they share the same input. The output neuron then forms the weighted sum $\hat{y} = \theta_0 + \theta_1 h_1 + \theta_2 h_2$. The centre diagram shows the same network with parameters omitted and the bias nodes drawn as dashed - a common convention indicating that bias inputs are always present but typically not drawn explicitly. The right diagram shows only the bare network topology. An important detail not visible in any of these diagrams: the output neuron o applies no activation function - it computes a plain weighted sum. Nonlinearity is introduced only in the hidden layer; the output is linear in the hidden activations.

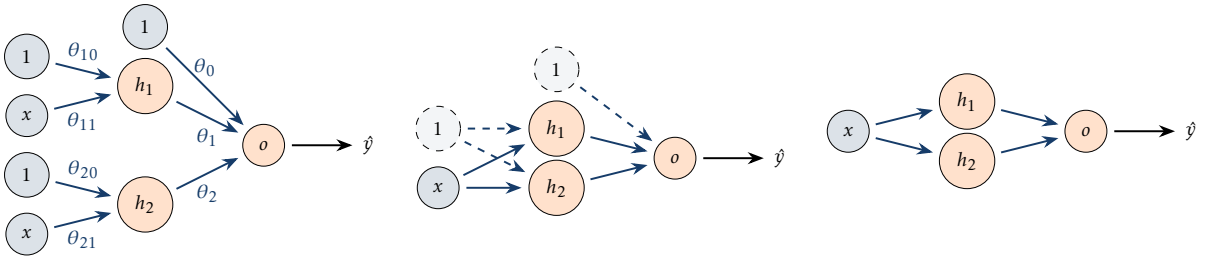

Figure 2.11: Three levels of abstraction for a two-neuron network. Left: full representation with all weights θ_{j1}, θ_{j0} and output weights labelled. Centre: simplified representation with bias nodes shown as dashed. Right: compact schematic showing topology only.

2.5. 1-Hidden-Layer Neural Networks

▸ Session 2: One Layer Neural Network

We now generalise from two hidden neurons to k hidden neurons, and from a single input to multiple inputs. The resulting architecture is a **one-hidden-layer neural network**, also called a **shallow neural network**.

With a single input $x \in \mathbb{R}$ and k hidden neurons, the network computes:

$$h_j = a(\theta_{j1}\, x + \theta_{j0}), \qquad j = 1, \ldots, k \tag{2.6}$$

$$\hat{y} = f_\theta(x) = \theta_0 + \sum_{j=1}^{k} \theta_j\, h_j. \tag{2.7}$$

Each hidden neuron j has its own parameters θ_{j1} (weight) and θ_{j0} (bias), and contributes one breakpoint to the piecewise linear output. The output neuron forms a weighted sum of all hidden activations with weights θ_j and a global bias θ_0. The graphical representation is shown in Figure 2.12.

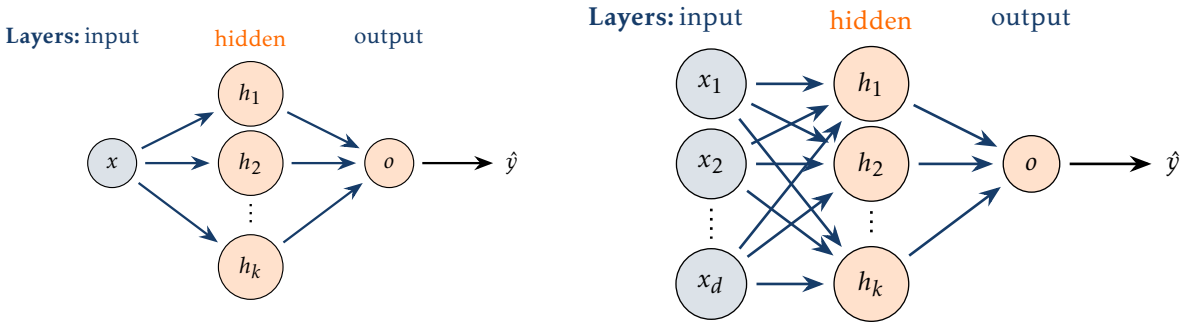

Figure 2.12: Left: one-hidden-layer network with a single input x and k hidden neurons. Right: extension to d inputs $\mathbf{x} = (x_1, \ldots, x_d)$. Each hidden neuron applies a ReLU activation; the output neuron computes a plain weighted sum.

Extending to d inputs $\mathbf{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$ is straightforward. Each hidden neuron now computes a linear combination of all inputs:

$$h_j = a\left(\sum_{i=1}^{d} \theta_{ji}\, x_i + \theta_{j0}\right), \qquad j = 1, \ldots, k \tag{2.8}$$

$$\hat{y} = f_\theta(\mathbf{x}) = \theta_0 + \sum_{j=1}^{k} \theta_j\, h_j. \tag{2.9}$$

The weight θ_{ji} connects input x_i to hidden neuron j, and θ_{j0} is the bias of neuron j. The output formula is unchanged — the hidden activations h_j are combined into a single prediction $\hat{y}$ regardless of the input dimensionality.

The collection of all parameters — the weights θ_{ji}, biases θ_{j0}, output weights θ_j, and output bias θ_0 — is denoted collectively as $\boldsymbol{\theta}$. Learning this network means finding the values of $\boldsymbol{\theta}$ that make $f_\theta(\mathbf{x}) \approx f^*(\mathbf{x})$ on the training data.

2.6. Universal Approximation

▸ Session 2: Universal Approximation

The piecewise linear intuition suggests that a shallow network with enough neurons can approximate any reasonable function. This is made precise by the **Universal Approximation Theorem**.

Theorem 1 (Universal Approximation (Cybenko 1989; Hornik 1991)). *For any continuous function $g : \mathcal{X} \to \mathbb{R}$ defined on a compact set $\mathcal{X} \subseteq \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a one-hidden-layer neural network f_θ such that*

$$\sup_{\mathbf{x}\in\mathcal{X}} |f_\theta(\mathbf{x}) - g(\mathbf{x})| < \varepsilon. \tag{2.10}$$

This result was first proved by Cybenko 1989 for sigmoid activations and later extended by Hornik 1991 to a broad class of activation functions. We do not prove this result here as it relies on advanced math concepts from measure theory and the Stone-Weierstrass theorem. The piecewise linear intuition developed in the previous section - more neurons, more breakpoints, better approximation - provides the geometric essence of why the theorem holds.

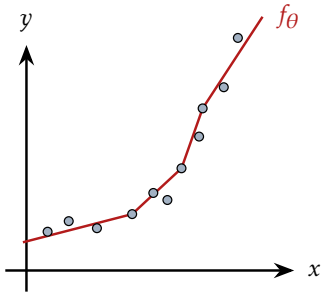

Figure 2.13: A piecewise linear function approximating a nonlinear curve.

The theorem is powerful: it says that the class of shallow neural networks is dense in the space of continuous functions. No matter what continuous function you want to approximate, a shallow network can get arbitrarily close. However, the theorem has important limitations that are easily overlooked. It guarantees existence - there *exists* a network that approximates g - but says nothing about how large that network must be, or how to find its parameters. In the worst case, exponentially many neurons may be required. The theorem also says nothing about generalisation: a network that perfectly approximates f^* on $\mathcal{X}$ may still fail on out-of-distribution inputs. Depth, which the theorem ignores, turns out to matter enormously in practice - deeper networks can represent the same functions far more efficiently than shallow ones, a point we return to later.

2.7. Binary Classification

▸ Session 2: Binary Classification

We now turn to the classification setting. In **binary classification**, the output space is $\mathcal{Y} = \{0, 1\}$, and the goal is to assign each input $\mathbf{x} \in \mathcal{X}$ to one of two classes.

The simplest approach is a **linear classifier**: predict class 1 if a linear function of the input exceeds zero, and class 0 otherwise:

$$\hat{y} = \mathbf{1}\left[\theta_0 + \sum_{i=1}^{d} \theta_i x_i \geq 0\right]. \tag{2.11}$$

The set of points satisfying $\theta_0 + \sum_i \theta_i x_i = 0$ is a hyperplane in $\mathbb{R}^d$ - the **decision boundary**. The weight vector $(\theta_1, \ldots, \theta_d)$ is normal to this hyperplane, and θ_0 controls its offset from the origin. In two dimensions, the decision boundary is a line; in three dimensions, a plane; in general, a $(d-1)$-dimensional hyperplane.

A linear classifier can only represent **linearly separable** problems - those in which the two classes can be separated by a single hyperplane. Many real problems are not linearly separable. A simple example: consider two clusters of one class located at the top-left and bottom-right of the input space, with a cluster of the other class in the middle. No straight line can separate them.

The solution is exactly analogous to the regression case: replace the linear function with a neural network. The decision boundary $f_\theta(\mathbf{x}) = 0$ becomes a nonlinear surface, capable of carving out arbitrarily complex regions of the input space:

$$\hat{y} = \mathbf{1}[f_\theta(\mathbf{x}) \geq 0]. \tag{2.12}$$

The same shallow network architecture used for regression - a linear combination of ReLU-activated linear functions - can serve as f_θ here. The universal approximation theorem applies equally to classification.

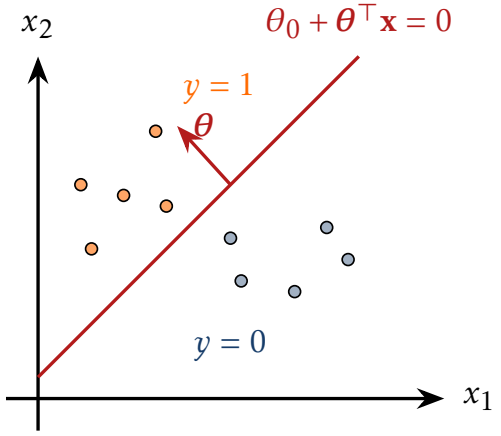

Figure 2.14: A linear classifier separates two classes with a hyperplane. The weight vector θ is normal to the decision boundary.

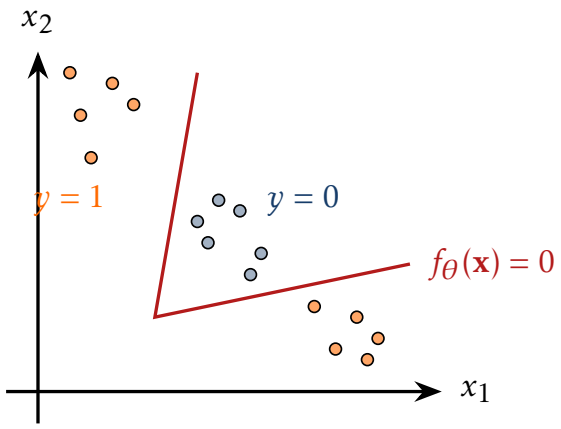

Figure 2.15: A nonlinearly separable dataset. A neural network with a nonlinear decision boundary $f_\theta(\mathbf{x}) = 0$ is required.

2.8. Biological vs Artificial Neuron

▹ Session 2: Biological vs Artificial Neuron

Having built up the neural network entirely from mathematical and computational considerations, it is worth pausing to examine the biological analogy that motivates the terminology.

A biological neuron illustrated in Figure 2.16 receives signals from other neurons via **dendrites**, integrates these signals in the **cell body** (soma), and fires an electrical pulse along its **axon** if the integrated signal exceeds a threshold. The artificial neuron in Figure 2.17 mimics this schematically: inputs $\mathbf{x}$ play the role of dendrites, the weighted sum $\sum_i \theta_i x_i + \theta_0$ plays the role of integration in the soma, and the activation function a plays the role of the firing threshold.

The analogy is useful as a narrative device but should not be taken literally. Real neurons are vastly more complex than the linear-threshold model: they operate in continuous time, communicate via spike trains rather than real-valued signals, exhibit rich temporal dynamics, and are embedded in three-dimensional physical structures that constrain their connectivity. The artificial neuron strips all of this away, retaining only the most abstract essence of the computation.

Indeed, the mathematical objects we have been working with - piecewise linear functions, weighted sums, nonlinear activations - were derived entirely from the regression and classification problems, without any reference to biology. The biological inspiration played a historical role in motivating the research programme, but the justification for using neural networks today is empirical and mathematical, not biological.

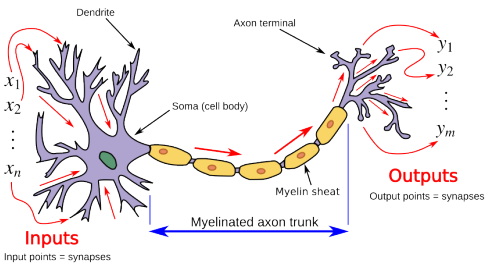

Image: Egm4313.s12 at English Wikipedia, CC BY-SA 3.0, via Wikimedia Commons

Figure 2.16: A biological neuron with dendrites, soma, and axon.

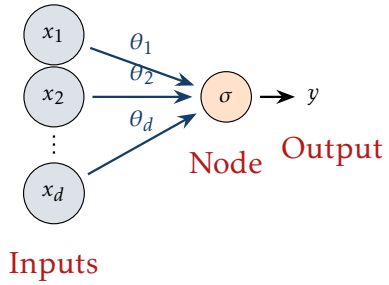

Figure 2.17: The artificial neuron: a weighted sum of inputs followed by a nonlinear activation function.

3. Deep Neural Networks & Training

▹ Session 3

These notes accompany the third lecture. In Session 2 we established that a one-hidden-layer network with ReLU activations can approximate any continuous function - but we stopped short of asking how to actually *find* good parameters. This session addresses that question. We first generalise the architecture to arbitrary depth, then introduce the machinery needed to train it: a loss function to measure error, and gradient descent to minimise it.

3.1. From Shallow to Deep Networks

▹ Session 3: From Shallow to Deep, Matrix Notation

The one-hidden-layer network from Session 2 computes, for input $\mathbf{x} \in \mathbb{R}^d$:

$$h_j = a\left(\theta_{j0} + \sum_{i=1}^{d} \theta_{ji}\, x_i\right), \qquad j = 1,\dots,k \tag{3.1}$$

$$\hat{y} = \theta_0 + \sum_{j=1}^{k} \theta_j\, h_j. \tag{3.2}$$

The natural extension is to stack additional layers: the output of one layer becomes the input of the next. Each layer applies a linear transformation followed by a nonlinear activation. Stacking layers allows the network to learn increasingly abstract representations - the first layer might detect simple patterns in the input, the second layer combinations of those patterns, and so on. Figure 3.1 contrasts the shallow network from Session 2 with a two-hidden-layer deep network.

To handle multiple layers cleanly, scalar notation becomes cumbersome. We start by collecting the individual quantities into vectors. The inputs $x_1,\dots,x_d$ are gathered into an input vector $\mathbf{x} \in \mathbb{R}^d$, and the hidden neuron activations $h_1,\dots,h_k$ into a hidden activation vector $\mathbf{h} \in \mathbb{R}^k$. Correspondingly, the weights and biases are collected into a matrix and a vector. The bias of each output neuron j gives a bias vector $\boldsymbol{\theta}_0 \in \mathbb{R}^k$, and the weights connecting all d input neurons to all k output neurons form a weight matrix $\boldsymbol{\Theta} \in \mathbb{R}^{k\times d}$:

$$\boldsymbol{\Theta} = \begin{pmatrix} \theta_{11} & \cdots & \theta_{1d} \\ \vdots & \ddots & \vdots \\ \theta_{k1} & \cdots & \theta_{kd} \end{pmatrix} \in \mathbb{R}^{k\times d}, \qquad \boldsymbol{\theta}_0 = \begin{pmatrix} \theta_{10} \\ \vdots \\ \theta_{k0} \end{pmatrix} \in \mathbb{R}^k. \tag{3.3}$$

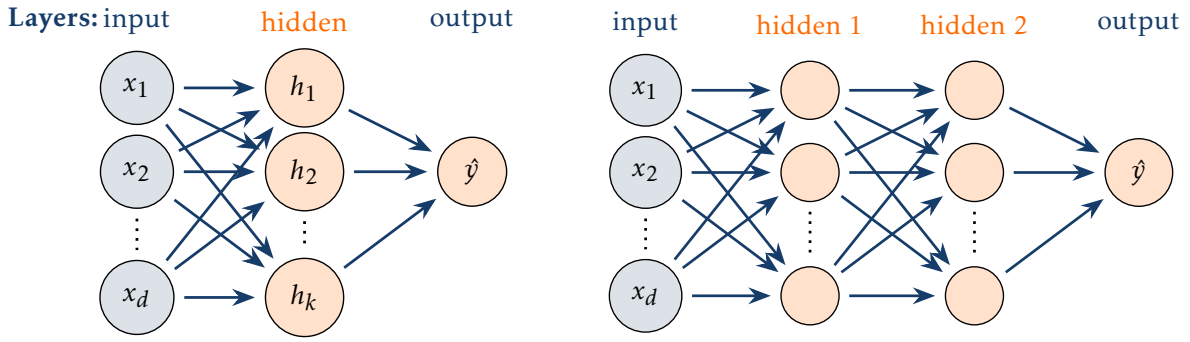

Figure 3.1: Left: shallow network with one hidden layer. Right: deep network with two hidden layers. Each additional layer introduces another linear transform followed by a nonlinearity.

The matrix $\mathbf{\Theta}$ has k rows and d columns. Each row j contains the weights $(\theta_{j1},\dots,\theta_{jd})$ connecting all d input neurons to output neuron j - so there is one row per output neuron. Each column i contains the weights $(\theta_{1i},\dots,\theta_{ki})$ connecting input neuron x_i to all k output neurons - so there is one column per input neuron. The bias vector $\boldsymbol{\theta}_0 \in \mathbb{R}^k$ has one entry per output neuron.

With this notation, the entire hidden layer computation collapses to a single vector equation:

$$\mathbf{h} = a(\boldsymbol{\theta}_0 + \mathbf{\Theta}\mathbf{x}) \in \mathbb{R}^k, \tag{3.4}$$

where a is applied elementwise. The matrix-vector product $\mathbf{\Theta}\mathbf{x} \in \mathbb{R}^{k\times d} \cdot \mathbb{R}^d = \mathbb{R}^k$ produces one pre-activation value per output neuron. Adding the bias $\boldsymbol{\theta}_0 \in \mathbb{R}^k$ and applying a elementwise gives the hidden activation vector $\mathbf{h} \in \mathbb{R}^k$.

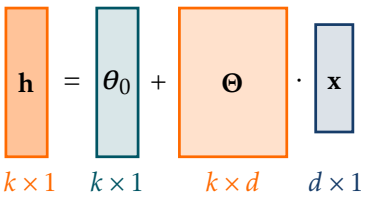

Figure 3.2: Hidden layer computation as matrix-vector operation.

3.2. Layer Notation and Function Composition

▸ Session 3: Layer Notation, Deep Network as Function Composition, Dimensions

We index layers $l = 0, 1, \dots, L$, where $l = 0$ denotes the input and $l = L$ the output. The forward pass is defined recursively:

$$\mathbf{h}^{(0)} = \mathbf{x}, \qquad \mathbf{h}^{(l)} = a\Big(\boldsymbol{\theta}_0^{(l)} + \mathbf{\Theta}^{(l)}\mathbf{h}^{(l-1)}\Big), \quad l = 1,\dots,L-1, \tag{3.5}$$

with output

$$\hat{y} = \boldsymbol{\theta}_0^{(L)} + \mathbf{\Theta}^{(L)}\mathbf{h}^{(L-1)}. \tag{3.6}$$

The output layer applies no activation function - the raw linear output $\hat{y}$ is passed directly as the network's output. This is the general convention for all network types: for regression, $\hat{y}$ is used directly as the prediction; for classification, $\hat{y}$ contains the raw logits to which a task-specific transformation (sigmoid, softmax) is applied afterwards, outside

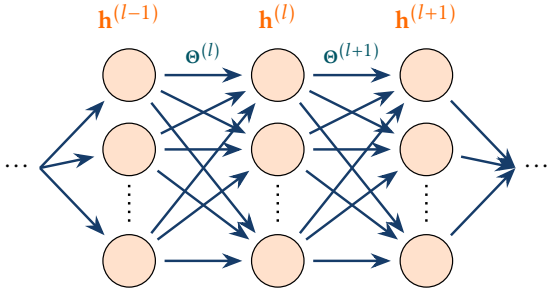

Figure 3.3: Layer notation.

the network. The layer structure is illustrated in Figure 3.3.

At layer l, the weight matrix has shape $\mathbf{\Theta}^{(l)} \in \mathbb{R}^{k_l \times k_{l-1}}$ and the bias vector $\boldsymbol{\theta}_0^{(l)} \in \mathbb{R}^{k_l}$, with $k_0 = d$. The matrix-vector product $\mathbf{\Theta}^{(l)}\mathbf{h}^{(l-1)} \in \mathbb{R}^{k_l \times k_{l-1}} \cdot \mathbb{R}^{k_{l-1}} = \mathbb{R}^{k_l}$ produces one pre-activation value per output neuron in layer l.

Viewed as a whole, the deep network is a composition of functions:

$$\hat{y} = f_\theta(\mathbf{x}) = \left(h_\theta^{(L)} \circ \cdots \circ h_\theta^{(1)}\right)(\mathbf{x}) = \boldsymbol{\theta}_0^{(L)} + \mathbf{\Theta}^{(L)}\, a\left(\cdots a\left(\boldsymbol{\theta}_0^{(1)} + \mathbf{\Theta}^{(1)}\mathbf{x}\right)\right). \tag{3.7}$$

The total number of parameters is the sum over all layers of the number of weights plus biases:

$$|\boldsymbol{\theta}| = \sum_{l=1}^{L} k_l \cdot (k_{l-1} + 1). \tag{3.8}$$

For example, the small network in Figure 3.4 with $d = 3$ inputs, two hidden layers of width $k_1 = k_2 = 4$, and $m = 2$ outputs has:

$$\begin{aligned} &4 \cdot (3+1) + 4 \cdot (4+1) + 2 \cdot (4+1) \\ &= 16 + 20 + 10 = 46 \text{ parameters.} \end{aligned} \tag{3.9}$$

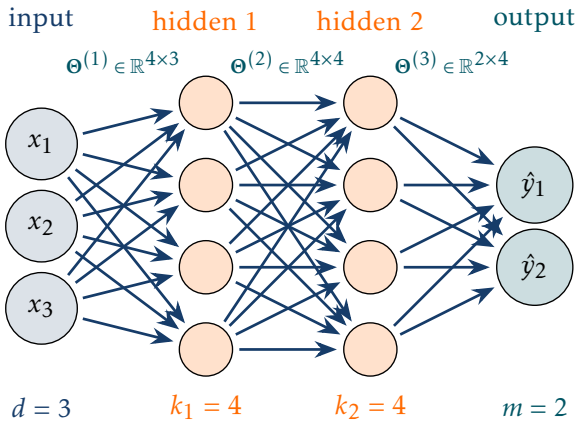

Figure 3.4: Network with $d = 3$, $k_1 = k_2 = 4$, $m = 2$. Total: $16 + 20 + 10 = 46$ parameters.

This is a toy example. In practice, networks are much wider and deeper - parameter counts grow quadratically with layer width and linearly with depth. A fully-connected network with $d = 784$ inputs (a 28×28 image), two hidden layers of width 1024, and 10 outputs already has approximately 1.8 million parameters. Modern architectures are far larger: ResNet-50, a standard image classification model, has 25 million parameters; GPT-2 has 1.5 billion; GPT-3 has 175 billion. Finding good values for all parameters jointly from data is the central challenge of deep learning.

3.3. Batch Formulation

▸ Session 3: Batch Formulation

In practice, we process multiple training samples simultaneously in a **batch**. Given N samples, we stack the inputs as rows of a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, and the hidden activations of layer l form a matrix $\mathbf{H}^{(l)} \in \mathbb{R}^{N \times k_l}$.

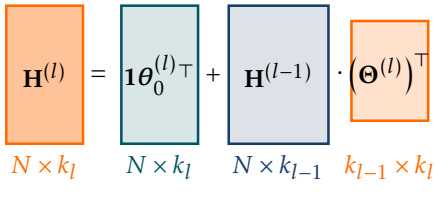

Figure 3.5: Batch forward pass.

The batch forward pass for layer l is:

$$\mathbf{H}^{(l)} = a\left(\mathbf{1}\boldsymbol{\theta}_0^{(l)\top} + \mathbf{H}^{(l-1)}\left(\mathbf{\Theta}^{(l)}\right)^\top\right), \tag{3.10}$$

where $\mathbf{1} \in \mathbb{R}^N$ broadcasts the bias across all samples, as illustrated in Figure 3.5. This is precisely

what PyTorch computes with `H = X @ W.T + b`: the batch dimension comes first, and the matrix multiply operates across the feature dimension.

Processing data in batches is computationally efficient: modern hardware (GPUs, TPUs) is designed for large parallel matrix operations, and a single batched matrix multiply is far faster than N sequential vector operations.

3.4. The Learning Problem

▹ Session 3: The Learning Problem

We now have a network f_θ and a dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$. The goal is to find parameters $\boldsymbol{\theta}$ such that $f_\theta(\mathbf{x}) \approx y$ on unseen data. This raises two immediate questions:

1. How do we **measure** how wrong f_θ is? $\Rightarrow$ *loss function*
2. How do we **find** good $\boldsymbol{\theta}$ systematically? $\Rightarrow$ *optimisation*

As illustrated in Figure 3.6, there are infinitely many functions that could plausibly fit the training data - they differ in how they interpolate between observed points and how they extrapolate beyond them. With a low-dimensional input we could perhaps inspect candidate functions visually. But f_θ maps from $\mathbb{R}^d$ where d may be hundreds or thousands - visual inspection is impossible. Trial and error is equally hopeless: the parameter space has millions of dimensions, and randomly trying parameter configurations would never converge to anything useful.

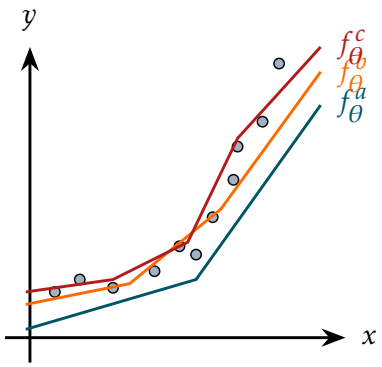

Figure 3.6: Many functions fit the data. Which is best?

What we need is a single scalar number that summarises how wrong the current f_θ is on the training data - small when the predictions are good, large when they are bad. This scalar is the **loss**, and it turns the vague goal of "find a good function" into a concrete mathematical optimisation problem: minimise the loss over all possible parameter settings.

3.5. Loss Functions

▹ Session 3: What is a Loss Function?, Regression: Mean Squared Error

A **loss function** $\ell(\hat{y}, y)$ measures the discrepancy between a prediction $\hat{y}$ and a true label y for a single sample. It satisfies $\ell(\hat{y}, y) = 0$ when $\hat{y} = y$ and $\ell(\hat{y}, y) > 0$ otherwise, growing larger as the prediction worsens. The loss is the compass that guides training: it converts the vague notion of "a good model" into a concrete scalar that can be minimised.

For a full dataset, we average the per-sample loss over all training examples to get the **dataset loss**:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell\Big(f_\theta(\mathbf{x}^{(i)}), y^{(i)}\Big). \tag{3.11}$$

Averaging rather than summing makes $\mathcal{L}$ independent of dataset size, which is convenient when comparing runs with different amounts of data or different batch sizes.

The choice of ℓ depends on the task. For regression we use Mean Squared Error; for binary classification, Binary Cross-Entropy; for multiclass classification, Cross-Entropy with a softmax output. Their probabilistic motivation via maximum likelihood

estimation is developed in a later session. Here we focus on MSE for regression; classification losses are covered when we discuss classification in detail.

3.5.1. Mean Squared Error

For regression with scalar output $y \in \mathbb{R}$ and prediction $\hat{y} \in \mathbb{R}$, the per-sample loss is the squared residual:

$$\ell(\hat{y}, y) = (\hat{y} - y)^2. \tag{3.12}$$

The dataset loss is accordingly:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{y}^{(i)} - y^{(i)}\right)^2. \tag{3.13}$$

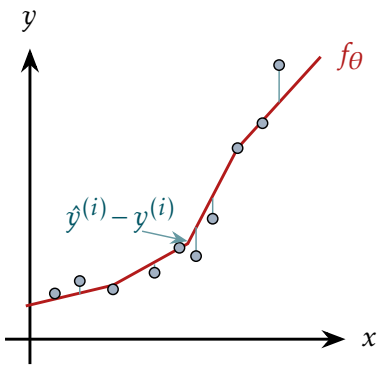

Figure 3.7: MSE loss. Vertical lines show residuals $\hat{y}^{(i)} - y^{(i)}$.

As shown in Figure 3.7, the residuals $\hat{y}^{(i)} - y^{(i)}$ are the vertical distances between the predicted function and the data points. MSE averages the squares of these distances. Three properties make squaring the natural choice. First, prediction errors can be positive or negative and would cancel if averaged directly - squaring removes the sign. Second, squaring penalises large errors more heavily than small ones: a prediction off by 10 contributes 100 times more to the loss than one off by 1. Third, the squared function is differentiable everywhere, which is essential for gradient-based optimisation.

3.6. Gradient Descent

▹ Session 3: The Optimization Problem, Gradient Descent, The Key Question

Training the network means finding parameters that minimise the dataset loss:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}). \tag{3.14}$$

Unlike linear regression, no closed-form solution exists: $\mathcal{L}(\boldsymbol{\theta})$ is highly nonlinear in $\boldsymbol{\theta}$ through the nested compositions of weight matrices and activations, the parameter space has millions of dimensions, and the loss surface is non-convex with many local minima and saddle points. We need an iterative strategy that navigates this landscape using only local information.

The key local information at any point $\boldsymbol{\theta}$ is the **gradient** $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$ - the vector of partial derivatives of $\mathcal{L}$ with respect to each parameter:

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \frac{\partial \mathcal{L}}{\partial \theta_2}, \dots \right). \tag{3.15}$$

The gradient points in the direction of steepest *ascent* of the loss surface. To *minimise* the loss we step in the opposite direction, giving the **gradient descent** update rule:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}), \tag{3.16}$$

where $\eta > 0$ is the **learning rate**, controlling the step size. Starting from a random initialisation, we repeatedly apply this update until the loss stops improving.

Figure 3.8 illustrates gradient descent on a one-dimensional quadratic loss $\mathcal{L}(\theta) = \theta^2$. Starting from $\theta_0 > 0$, the gradient $\frac{\partial \mathcal{L}}{\partial \theta} = 2\theta$ is positive, so the update $\theta \leftarrow \theta - \eta \cdot 2\theta$ moves θ to the left, towards the minimum at $\theta^* = 0$. As θ decreases, the gradient decreases too, so subsequent steps are naturally smaller even with a fixed learning rate — the algorithm automatically slows down as it approaches the optimum.

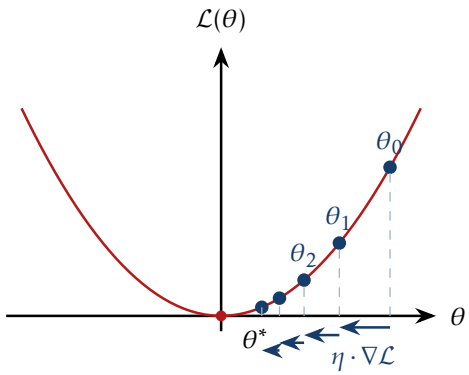

Figure 3.8: Gradient descent on a quadratic loss. Steps from θ_0 towards minimum θ^*.

The learning rate η is a critical hyperparameter. Too large and the updates overshoot the minimum, bouncing back and forth or even diverging. Too small and convergence is impractically slow. In practice, η is carefully tuned and often adapted during training - a topic we return to in the dedicated optimisation session.

The central computational challenge is evaluating $\nabla_\theta \mathcal{L}(\theta)$ efficiently. The loss has millions of components in θ, and computing each partial derivative $\frac{\partial \mathcal{L}}{\partial \theta_j}$ separately would require one forward pass per parameter - completely intractable. The answer is **backpropagation**: an algorithm that computes all partial derivatives simultaneously in a single backward pass, at a cost comparable to a single forward pass. We develop it in the next session.

4. Computation Graphs & Backpropagation

▹ Session 4

These notes accompany the fourth lecture. The central question left open at the end of Session 3 was: how do we compute $\nabla_\theta \mathcal{L}(\theta)$ efficiently for a deep network? The answer - backpropagation - is the algorithmic backbone of all modern deep learning. We develop it here through the framework of computation graphs.

4.1. Forward and Backward Pass

▹ Session 4: Forward and Backward Pass

Training a deep network requires two passes through the network for each update step. The **forward pass** evaluates the network layer by layer from input to output, computing the prediction $\hat{y}$ and then the loss $\mathcal{L}$:

$$\mathbf{x} = \mathbf{h}^{(0)} \xrightarrow{\boldsymbol{\Theta}^{(1)}} \mathbf{h}^{(1)} \xrightarrow{\boldsymbol{\Theta}^{(2)}} \cdots \xrightarrow{\boldsymbol{\Theta}^{(L)}} \mathbf{h}^{(L)} = \hat{y} \xrightarrow{\ell} \mathcal{L}. \tag{4.1}$$

The **backward pass** propagates gradients in the reverse direction, from the loss back through each layer to the parameters:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} \xleftarrow{\boldsymbol{\Theta}^{(1)}} \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \xleftarrow{\boldsymbol{\Theta}^{(2)}} \cdots \xleftarrow{\boldsymbol{\Theta}^{(L)}} \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \xleftarrow{\ell} \mathcal{L}. \tag{4.2}$$

The **gradient** $\nabla_\theta \mathcal{L}(\theta) = \frac{\partial \mathcal{L}}{\partial \theta}$ collects one partial derivative per parameter. Backpropagation is the algorithm that computes this backward pass efficiently. It is important to note that backpropagation *computes gradients* - it does not perform any optimisation. The gradient descent update step is separate.

4.2. Computation Graphs

▸ Session 4: Computation Graphs - Motivation, Example, Flexible Level of Detail, Flexible Operations, Scalar vs Vector

A **computation graph** is a directed acyclic graph in which each node represents an elementary operation and each edge represents the flow of data between operations. Decomposing a complex function into a computation graph makes gradient computation tractable for two reasons. First, each atomic operation has a simple, easy-to-compute local derivative. Second, intermediate values computed during the forward pass can be stored and reused during the backward pass, avoiding redundant computation.

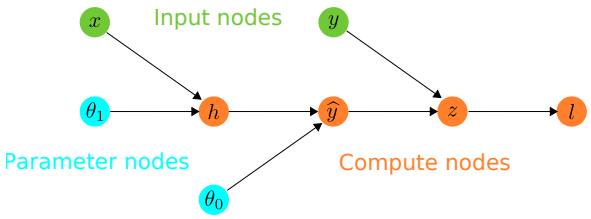

Figure 4.1: Computation graph.

A computation graph has three types of nodes:

- **Input nodes** - hold fixed data values (x, y). They store a value but have no gradient.
- **Parameter nodes** - hold learnable parameters ($\boldsymbol{\theta}$). They store a value and accumulate a gradient.
- **Compute nodes** - implement elementary operations. They store a value, accumulate a gradient, and know how to compute both their forward function and their local gradient.

4.2.1. Example: Linear Regression

Consider linear regression with MSE loss: $\ell(x, y, \boldsymbol{\theta}) = (\theta_0 + \theta_1 x - y)^2$. Figure 4.1 shows the computation graph. We decompose this into four atomic steps: (1) $h = \theta_1 x$, (2) $\hat{y} = \theta_0 + h$, (3) $z = \hat{y} - y$, (4) $\ell = z^2$. Each step is a simple operation - multiplication, addition, subtraction, squaring - with a derivative that can be written down immediately.

4.2.2. Flexible Level of Detail

The same function can be decomposed at different levels of granularity, as illustrated in Figure 4.2. The four-step decomposition above is fine-grained; we could also use a coarser two-step decomposition: (1) $\hat{y} = \theta_0 + \theta_1 x$, (2) $\ell = (\hat{y} - y)^2$, or a vectorised version with $\hat{y} = \boldsymbol{\theta}^\top \mathbf{x}$. The choice of granularity is a design decision: finer decompositions make each local gradient simpler, while coarser ones reduce the number of nodes and may be more efficient if the compound derivative is known analytically.

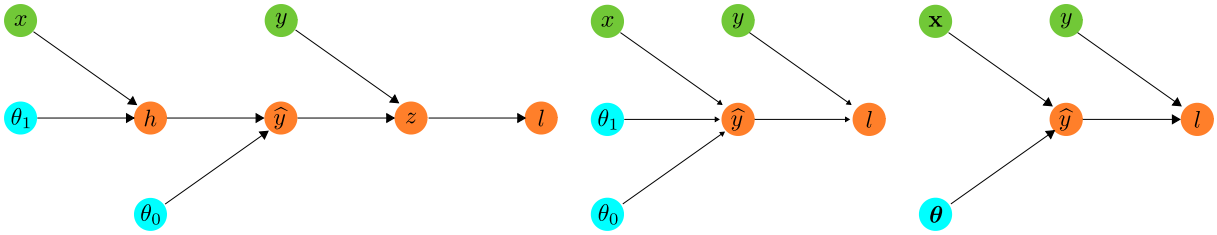

Figure 4.2: Three levels of decomposition for the same function $\ell = (\theta_0 + \theta_1 x - y)^2$. Left: fine-grained (4 nodes). Centre: coarser (2 nodes). Right: vectorised.

4.2.3. Flexible Operations

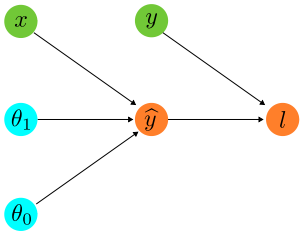

Figure 4.3: Same graph, different node functions.

The same graph structure can represent different models by changing the node functions. As shown in Figure 4.3, a two-node graph covers both linear regression with MSE and logistic regression with BCE - only the functions at each node differ.

Linear regression + MSE: (1) $\hat{y} = \theta_0 + \theta_1 x$, (2) $\ell = (\hat{y} - y)^2$

Logistic regression + BCE: (1) $\hat{y} = \sigma(\theta_0 + \theta_1 x)$, (2) $\ell = -y \log \hat{y} - (1-y)\log(1-\hat{y})$

This modularity is the foundation of modern deep learning frameworks, which implement a library of differentiable operations that can be composed freely.

4.2.4. Scalar vs Vector Notation

Computation graphs can be drawn at the level of scalar operations - one node per neuron, one edge per weight - or at the level of vector operations - one node per layer, as shown in Figure 4.4. Scalar graphs make the gradient flow explicit; vector graphs are more compact and correspond directly to the matrix operations implemented in practice. The two representations compute the same quantities; the vector form simply groups many scalar operations into a single matrix node.

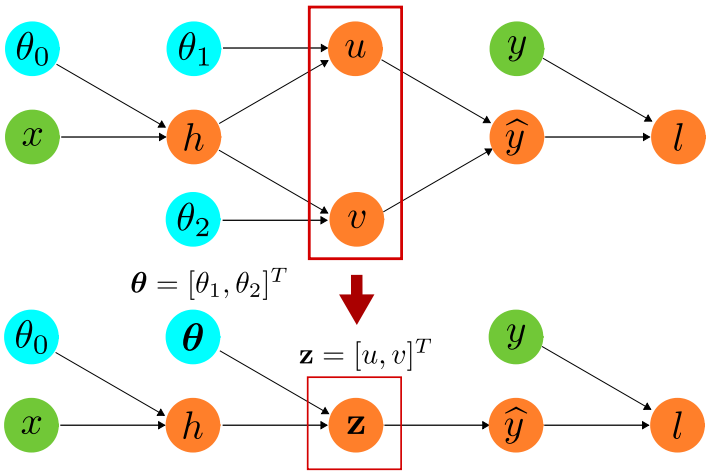

Figure 4.4: Scalar (top) and vector (bottom) computation graphs.

4.3. Backpropagation

Backpropagation relies on two rules from calculus.

▸ Session 4: Two Key Rules, Local vs Global Gradients

Remark 1 (Leibniz notation). We use **Leibniz notation**: $\frac{df}{dx}$ denotes the derivative of f with respect to x, and $\frac{\partial f}{\partial x}$ the partial derivative when f depends on multiple variables. This is equivalent to the prime notation $f'(x)$ but makes the variable of differentiation explicit, which is essential when tracking gradients through multiple composed functions.

Sum rule. The gradient of an average is the average of the gradients:

$$\nabla_\theta \mathcal{L}(\theta) = \nabla_\theta \frac{1}{N} \sum_{i=1}^{N} \ell(\mathbf{x}^{(i)}, y^{(i)}, \theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \ell(\mathbf{x}^{(i)}, y^{(i)}, \theta). \tag{4.3}$$

This has an important practical consequence: we only need to work out how to compute the gradient for a *single* training example $(\mathbf{x}^{(i)}, y^{(i)})$. Once we know $\nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$ for one example, the gradient for the full dataset is simply the average of these individual gradients. This means the entire complexity of backpropagation reduces to differentiating through the network for a single sample.

Chain rule. For a composed function $f(g(x))$:

$$\frac{d}{dx} f(g(x)) = \frac{df}{dg}\frac{dg}{dx}. \tag{4.4}$$

When a variable x feeds into multiple downstream nodes $g_1(x), \ldots, g_m(x)$, the chain rule generalises to:

$$\frac{d}{dx} f(g_1(x), \ldots, g_m(x)) = \sum_{i=1}^{m} \frac{\partial f}{\partial g_i}\frac{dg_i}{dx}. \tag{4.5}$$

The sum arises because x influences f through multiple paths; the total effect is the sum of contributions from each path.

The chain rule is the engine of backpropagation: it tells us that to differentiate through a composition of functions, we multiply local derivatives layer by layer. This multiplication of many terms has a crucial consequence for deep networks - gradients can grow or shrink exponentially as they are propagated backwards through many layers. When they shrink towards zero, parameters in early layers receive negligible gradient signal and fail to learn (**vanishing gradients**); when they grow without bound, training becomes unstable (**exploding gradients**). These phenomena and the techniques used to address them are developed in a later session.

4.3.1. Local and Global Gradients

At each node in the computation graph we distinguish two quantities:

- The **local gradient** $\frac{\partial \text{out}}{\partial \text{in}}$ is the derivative of the node's output with respect to its input. It depends only on the local computation at that node, and can be computed during the forward pass from the stored input values.
- The **global gradient** $\frac{\partial \ell}{\partial \text{in}}$ is the derivative of the loss with respect to that node's input. It accumulates contributions from all paths leading from that node to the loss.

The backpropagation update at each node is:

$$\frac{\partial \ell}{\partial \text{in}} = \frac{\partial \ell}{\partial \text{out}} \cdot \frac{\partial \text{out}}{\partial \text{in}}. \tag{4.6}$$

In words: multiply the incoming global gradient (flowing in from the right) by the local gradient to obtain the outgoing global gradient (flowing to the left). This is applied at every node, proceeding right to left through the graph.

4.4. Worked Examples

4.4.1. Linear Regression - Symbolic

▸ Session 4: Backprop Example: Linear Regression (Symbolic), (Numerical), Logistic Regression

We work through the backpropagation pass for linear regression using the four-step decomposition, illustrated in Figure 4.5. The three columns below correspond to the

three passes through the graph: forward pass and local gradients are evaluated left to right through the graph, reading top to bottom; global gradients are accumulated right to left, reading bottom to top.

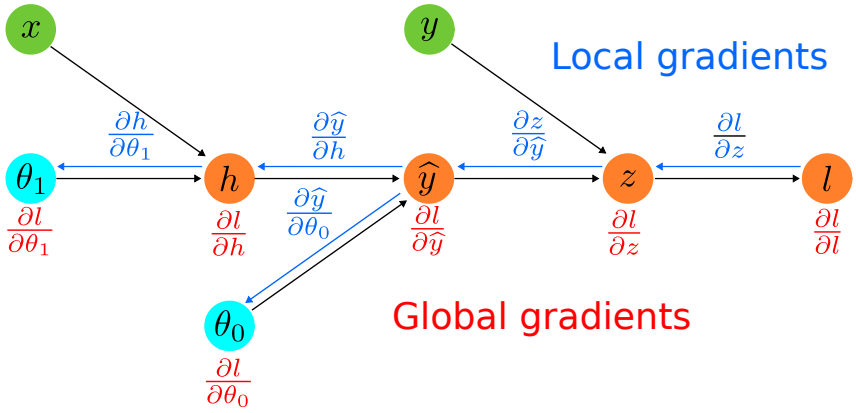

Figure 4.5: Symbolic local (blue) and global (red) gradients on the linear regression graph.

Forward pass ↓	**Local gradients** ↓	**Global gradients** ↑
	(1) $\partial h/\partial \theta_1 = x$	(4) $\partial \ell/\partial \theta_1 = \partial \ell/\partial h \cdot x$
(1) $h = \theta_1 x$	(2) $\partial \hat{y}/\partial \theta_0 = 1$	$\partial \ell/\partial \theta_0 = \partial \ell/\partial h \cdot 1$
(2) $\hat{y} = \theta_0 + h$	$\partial \hat{y}/\partial h = 1$	(3) $\partial \ell/\partial h = \partial \ell/\partial \hat{y} \cdot 1$
(3) $z = \hat{y} - y$	(3) $\partial z/\partial \hat{y} = 1$	(2) $\partial \ell/\partial \hat{y} = \partial \ell/\partial z \cdot 1$
(4) $\ell = z^2$	(4) $\partial \ell/\partial z = 2z$	(1) $\partial \ell/\partial z = 1 \cdot 2z$

The gradients with respect to the parameters θ_0 and θ_1 are what gradient descent requires.

4.4.2. Linear Regression - Numerical

With concrete values $x = 2$, $y = 5$, $\theta_0 = 1$, $\theta_1 = 3$, Figure 4.6 shows the forward values and gradients on the graph.

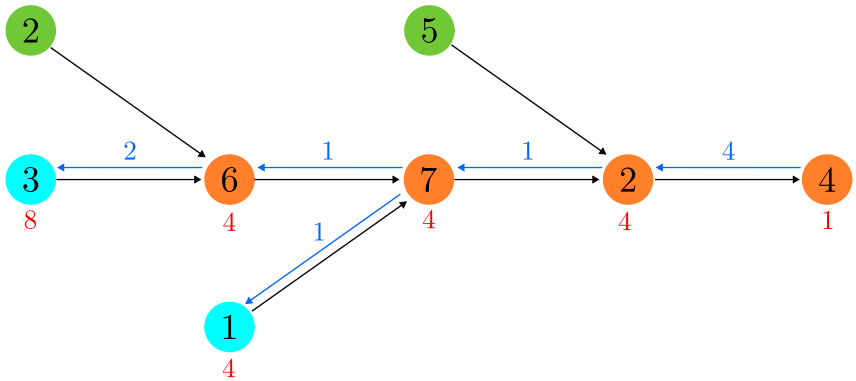

Figure 4.6: Numerical values on the computation graph for $x = 2$, $y = 5$, $\theta_0 = 1$, $\theta_1 = 3$.

Forward pass ↓	**Local gradients ↓**	**Global gradients ↑**
	(1) $\partial h/\partial \theta_1 = 2$	(4) $\partial \ell/\partial \theta_1 = 4 \cdot 2 = 8$
(1) $h = 3 \cdot 2 = 6$	(2) $\partial \hat{y}/\partial \theta_0 = 1$	$\partial \ell/\partial \theta_0 = 4 \cdot 1 = 4$
(2) $\hat{y} = 1 + 6 = 7$	$\partial \hat{y}/\partial h = 1$	(3) $\partial \ell/\partial h = 4 \cdot 1 = 4$
(3) $z = 7 - 5 = 2$	(3) $\partial z/\partial \hat{y} = 1$	(2) $\partial \ell/\partial \hat{y} = 4 \cdot 1 = 4$
(4) $\ell = 2^2 = 4$	(4) $\partial \ell/\partial z = 4$	(1) $\partial \ell/\partial z = 1 \cdot 4 = 4$

The backward pass gives $\partial \ell/\partial \theta_0 = 4$ and $\partial \ell/\partial \theta_1 = 8$, which are the gradients used to update the parameters.

4.4.3. Logistic Regression

The same graph structure as in Figure 4.5 applies to logistic regression with sigmoid activation and binary cross-entropy loss — only the node functions differ. This is the key insight: backpropagation is a general algorithm that operates on the graph, independent of what functions the nodes implement.

Forward pass ↓	**Local gradients ↓**	**Global gradients ↑**
	(1) $\partial h/\partial \theta_0 = 1$	(3) $\partial \ell/\partial \theta_0 = \partial \ell/\partial h \cdot 1$
(1) $h = \theta_0 + \theta_1 x$	$\partial h/\partial \theta_1 = x$	$\partial \ell/\partial \theta_1 = \partial \ell/\partial h \cdot x$
(2) $\hat{y} = \sigma(h)$	(2) $\partial \hat{y}/\partial h = \sigma(h)(1-\sigma(h))$	(2) $\partial \ell/\partial h = \partial \ell/\partial \hat{y} \cdot \partial \hat{y}/\partial h$
(3) $\ell = -y \log \hat{y}$	(3) $\partial \ell/\partial \hat{y} = \dfrac{\hat{y} - y}{\hat{y}(1-\hat{y})}$	(1) $\partial \ell/\partial \hat{y} = 1 \cdot \dfrac{\hat{y} - y}{\hat{y}(1-\hat{y})}$
$\quad -(1-y)\log(1-\hat{y})$		

4.5. Multiple Neurons and Gradient Accumulation

▸ Session 4: Backprop: Multiple Neurons in a Layer

When a node's output feeds into multiple downstream nodes, as in Figure 4.7, the chain rule generalisation applies: the global gradient at that node is the *sum* of contributions from each downstream path. Concretely, if node h feeds into nodes u and v:

$$\frac{\partial \ell}{\partial h} = \frac{\partial \ell}{\partial u} \cdot \frac{\partial u}{\partial h} + \frac{\partial \ell}{\partial v} \cdot \frac{\partial v}{\partial h}. \tag{4.7}$$

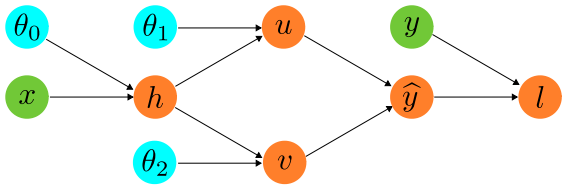

Figure 4.7: A node h feeding into two downstream nodes u and v. Gradients accumulate from both paths.

In implementation, gradients from all downstream paths are simply summed: each downstream node contributes its share, and the total gradient at the shared upstream node is their sum.

4.6. Technical Implementation and Vectorisation

▸ Session 4: Technical Implementation, From For Loops to Vectorization

Each node stores two quantities: `*.val` (the forward value) and `*.grad` (the accumulated gradient), with input nodes storing only `*.val`. The forward pass initialises all `*.val` by evaluating nodes from left to right. The backward pass initialises all `*.grad`

to zero, sets `l.grad = 1` (the seed), and then accumulates gradients from right to left using `grad += incoming_global * local_grad`.

In practice, the scalar for-loop implementation is far too slow for large networks. The key optimisation is **vectorisation**: replace loops over individual neurons with matrix operations. For a single layer:

```
Forward:  z.val = W.val @ x.val + b.val
          a.val = act(z.val)
Backward:  z.grad = a.grad * dact(z.val)
           W.grad = z.grad[:, None] * x.val[None, :].        (4.8)
```

The vectorised version computes the same quantities as the for-loop, but exploits the parallelism of modern hardware. On a GPU, this can be $\sim$100$\times$ faster.

5. Optimization

▸ Session 5

These notes accompany the fifth lecture. Sessions 3 and 4 established the two ingredients needed for training: a loss function $\mathcal{L}(\boldsymbol{\theta})$ that measures how wrong the network is, and backpropagation to compute its gradient $\nabla_{\boldsymbol{\theta}}\mathcal{L}(\boldsymbol{\theta})$ efficiently. What we have not yet addressed is how to *use* those gradients to find good parameters. That is the subject of this session. We develop the theory of gradient-based optimisation for deep networks: starting from vanilla gradient descent, motivating the need for stochastic approximations, then building up to the adaptive methods that are standard in practice. We also discuss two practical prerequisites that are often underemphasised: input normalisation and weight initialisation, both of which have a direct and significant effect on whether optimisation succeeds at all.

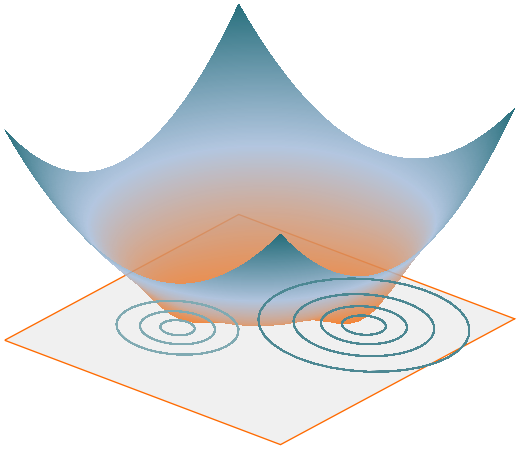

Figure 5.1: A non-convex loss surface over two parameters. Two distinct basins are visible, each with its own minimum. The contours projected on the base plane reveal their shape.

5.1. Optimisation Problem

▸ Session 5: Where We Are, Optimization Problem

Training a network means solving:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \arg\min_{\boldsymbol{\theta}} \frac{1}{N}\sum_{i=1}^{N} \ell\Big(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}\Big). \tag{5.1}$$

Remark 2. It is worth distinguishing min from arg min. The expression $\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$ denotes the smallest value the loss attains — a scalar. The expression $\arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$ denotes the parameter vector at which that value is attained — an element of the parameter space. As a simple example, $\min_{\theta}(\theta-3)^2 = 0$, whereas $\arg\min_{\theta}(\theta-3)^2 = 3$. In training we want $\boldsymbol{\theta}^*$ itself, since it is the parameters we will deploy at test time,

not the loss value they achieve.

This is a high-dimensional, non-convex optimisation problem. The parameter vector $\boldsymbol{\theta}$ may have millions of components; the loss surface has no closed-form minimum; and the landscape is riddled with local minima, saddle points, and flat regions. Figure 5.1 illustrates a simple two-parameter loss surface with two distinct basins. Unlike linear regression, no analytical solution exists — we must resort to iterative methods.

5.2. Gradient Descent

▸ Session 5: Gradient Descent

The fundamental algorithm is **gradient descent**. Starting from an initial parameter vector $\boldsymbol{\theta}_0$, we repeatedly apply the update:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}), \tag{5.2}$$

where $\eta > 0$ is the **learning rate**. Since the gradient points in the direction of steepest ascent of the loss, stepping opposite to it moves us downhill. As illustrated in Figure 3.8 from Session 3, the iterates move toward the minimum with steps that naturally decrease as the gradient shrinks near the optimum. Algorithm 1 states this formally.

Algorithm 1: Gradient Descent

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, learning rate η, initial parameters $\boldsymbol{\theta}_0$
for $t = 1, 2, \ldots$ *until convergence* **do**
  forward: $f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ for $i = 1, \ldots, N$
  loss: $\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$
  backprop: $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D})$
  update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D})$
end

5.2.1. Loss Landscape

The loss surface of a deep network is a function from the parameter space — potentially millions of dimensions — to a scalar. Understanding its geometry is essential for understanding why gradient descent behaves the way it does. Figure 5.2 shows a one-dimensional cross-section illustrating the four main features encountered in practice.

Local minima are points where the gradient is zero and the loss curves upward in every direction — gradient descent cannot escape them by following the gradient. A common concern is that deep networks have many local minima and training will get stuck in a bad one. In practice this turns out to be less of a problem than feared: empirical evidence suggests that local minima in deep networks tend to have loss values close to the global minimum, and the network performs well regardless of which one is found.

Saddle points are zero-gradient points where the loss curves upward in some di-

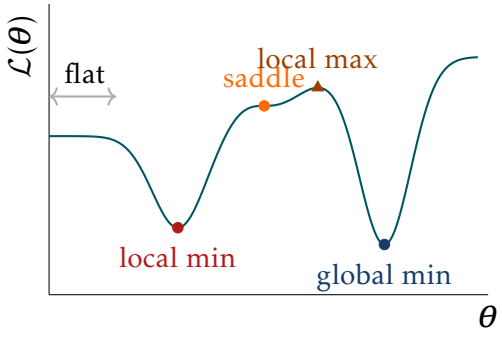

Figure 5.2: A non-convex loss landscape: local minimum, local maximum, saddle point (inflection), and flat region.

rections and downward in others. In a one-dimensional cross-section they appear as inflection points — where the curvature changes sign — with a near-zero gradient. In high dimensions, saddle points are far more common than local minima: a random critical point in a high-dimensional landscape is overwhelmingly likely to be a saddle rather than a true minimum. They are the more serious obstacle for gradient descent, because the gradient can be very small without the iterate being near a good solution.

Flat regions are extended areas where the gradient is very small but not exactly zero. Gradient descent makes negligible progress here — steps are tiny and many iterations are wasted. Flat regions often appear early in training when the network has not yet learned anything useful, or late in training when the network is near but not at a minimum.

Cliffs are sharp changes in the loss surface where the gradient is very large. A single gradient step in a cliff region can move the parameters by a huge amount, potentially undoing many previous steps. This is particularly common in recurrent networks and very deep architectures, and is the motivation for gradient clipping, which we will revisit in a later session.

The overarching message is that the loss landscape of a deep network is highly non-convex and has complex geometry. This means there is no guarantee that gradient descent will find the global minimum — or even a good local minimum — from an arbitrary starting point. The choice of initialisation, learning rate, and optimiser all affect which part of the landscape is explored and what solution is ultimately found.

5.2.2. Gradient Descent on Loss Landscape

▸ Session 5: Gradient Descent on Loss Landscape

An important practical point is that the loss surface is not known in advance — gradient descent probes it one point at a time. The algorithm starts from a random initialisation and uses local gradient information to decide where to step next. It has no global view of the landscape.

The step size — determined by the learning rate η — creates a fundamental tension. A small η leads to slow, expensive progress; a large η causes the iterates to jump erratically, potentially missing minima or diverging altogether. The choice of η is therefore critical and will be revisited when we discuss learning rate schedules.

Crucially, which minimum is found depends on the initialisation. Figure 5.3 illustrates this: two trajectories starting from nearby points on either side of a local maximum end up in completely different minima. This sensitivity to initialisation is not merely a theoretical curiosity — it has direct practical consequences, which is why weight initialisation deserves careful attention.

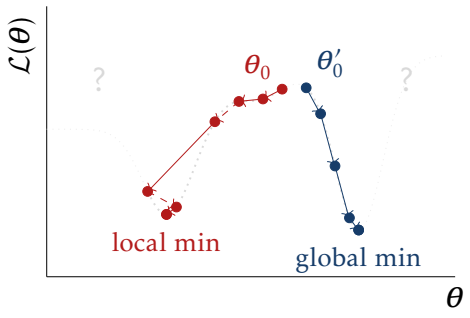

Figure 5.3: Two gradient descent trajectories starting from nearby initialisations on either side of a local maximum.

5.3. Stochastic and Mini-Batch Gradient Descent

▸ Session 5: The Cost of Full-Batch GD, Batch Mini-Batch and SGD, The Training Loop

5.3.1. Cost of Full-Batch Gradient Descent

Full-batch gradient descent as stated in Algorithm 1 computes the gradient over the entire dataset at each step. For datasets with millions of examples this is prohibitively expensive: one gradient evaluation requires one forward and one backward pass over all N samples.

The key observation is that the full loss is an average over individual sample losses:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}). \tag{5.3}$$

A random subset $\mathcal{B} \subset \{1, \ldots, N\}$, called a **mini-batch**, gives an unbiased estimate of the full loss:

$$\hat{\mathcal{L}}(\boldsymbol{\theta}; \mathcal{B}) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}), \qquad \mathbb{E}_{\mathcal{B}}\Big[\hat{\mathcal{L}}(\boldsymbol{\theta}; \mathcal{B})\Big] = \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}). \tag{5.4}$$

Since the gradient is a linear operator, an unbiased estimate of the loss immediately gives an unbiased estimate of the gradient:

$$\mathbb{E}_{\mathcal{B}}\Big[\nabla_{\boldsymbol{\theta}} \hat{\mathcal{L}}(\boldsymbol{\theta}; \mathcal{B})\Big] = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}; \mathcal{D}). \tag{5.5}$$

This justifies replacing the exact gradient with its mini-batch estimate in the update rule (5.2). In expectation, the update moves in the same direction as full-batch gradient descent; in practice, each step introduces some noise from the random batch selection.

Remark 3. The notation $\mathbb{E}_{\mathcal{B}}[\cdot]$ denotes the **expected value** over all possible mini-batches $\mathcal{B}$ drawn uniformly at random from $\mathcal{D}$. Concretely, if every subset of size B is equally likely, then $\mathbb{E}_{\mathcal{B}}[\hat{\mathcal{L}}]$ is the average of $\hat{\mathcal{L}}(\boldsymbol{\theta}; \mathcal{B})$ over all such subsets.

An estimator $\hat{\mathcal{L}}$ is called **unbiased** for $\mathcal{L}$ if $\mathbb{E}_{\mathcal{B}}[\hat{\mathcal{L}}] = \mathcal{L}$ — that is, averaged over all possible randomly drawn batches, the mini-batch loss equals the full dataset loss. Any individual batch estimate will typically differ from the true loss: depending on which samples happen to be selected, $\hat{\mathcal{L}}$ may be higher or lower than $\mathcal{L}$. What unbiasedness guarantees is that these fluctuations are symmetric and cancel out on average — there is no systematic tendency to over- or under-estimate the true loss.

This connects directly to the iterative nature of training. Over many gradient steps, each using a freshly drawn mini-batch, we are effectively computing many independent estimates of the gradient and acting on each in turn. Even though each individual step is noisy, the cumulative effect of many unbiased steps approximates what many steps with the exact gradient would achieve. The algorithm never computes the average explicitly — it simply takes many steps, and the averaging happens implicitly across iterations.

5.3.2. Batch Size and SGD Spectrum

The batch size $|\mathcal{B}| = B$ interpolates between two extremes. When $B = N$ we recover full-batch gradient descent: each update uses the exact gradient, convergence is smooth, but each step requires a full pass through the data — prohibitively expensive for large datasets. When $B = 1$ we obtain **stochastic gradient descent** (SGD): each step uses

the gradient of a single sample, making updates maximally cheap but also maximally noisy. Despite the noise, SGD has solid theoretical convergence guarantees and was the method of choice before GPU computing made larger batches practical.

Between these extremes lies **mini-batch SGD**, which introduces a controlled amount of noise into the gradient estimates. This noise turns out to have a beneficial effect on how the optimiser navigates the loss landscape. Because each mini-batch is a different random subset of the data, consecutive gradient steps are computed on slightly different loss surfaces. A sharp, narrow minimum that looks like a minimum on one batch may not look like one on the next — the iterate gets perturbed away from it. A wide, flat minimum, by contrast, looks like a minimum for essentially every batch, so the iterate settles there and stays. Over the course of training, this stochastic perturbation acts as a kind of implicit regularisation: the optimiser tends to drift away from sharp minima and toward flat ones. This matters because flat minima generalise better — the loss does not change much when the parameters shift slightly, which is exactly what happens when moving from training data to unseen test data.

The same mechanism helps with saddle points. At a saddle point, the full-batch gradient is near zero — the optimiser would stall. But the mini-batch gradient is not zero: the random selection of samples breaks the cancellation that produces the near-zero full-batch gradient, providing a noisy push that can carry the iterate out of the saddle region.

The practical default is mini-batch SGD with B typically between 32 and 512. The choice of batch size involves several competing considerations. Larger batches give more accurate gradient estimates, leading to more stable updates and smoother convergence — but each step is more expensive and fewer steps can be taken per unit of compute, and the landscape-navigation benefits of noise are reduced. Smaller batches are cheaper per step and explore the landscape more aggressively, but the high variance means individual steps are less informative and the optimiser can struggle to make steady progress. There is also a hardware dimension: modern GPUs are designed for large parallel matrix operations and are underutilised by very small batches, so batch sizes are typically chosen as powers of two (32, 64, 128, 256) to align with GPU memory layouts. In practice, batch size is treated as a hyperparameter and tuned alongside the learning rate — the two are closely coupled, since a larger batch typically calls for a proportionally larger learning rate to make comparable progress per epoch.

5.3.3. Training Loop

A complete training run iterates over the dataset multiple times. One full pass through the dataset is called an **epoch**; one mini-batch update is called an **iteration**. Algorithm 2 states the full training loop.

Algorithm 2: Mini-Batch Gradient Descent

Input: Dataset $\mathcal{D}$, learning rate η, initial parameters $\boldsymbol{\theta}_0$, number of epochs T
for *epoch* $= 1, \ldots, T$ **do**
  shuffle $\mathcal{D}$ into random mini-batches $\mathcal{B}_1, \mathcal{B}_2, \ldots$
  for *each mini-batch* $\mathcal{B}$ **do**
    forward: compute $f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ for $i \in \mathcal{B}$
    loss: $\hat{\mathcal{L}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell_i$
    backward: compute $\nabla_{\boldsymbol{\theta}} \hat{\mathcal{L}}$ via backpropagation
    update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \hat{\mathcal{L}}$
  end
end

Shuffling the dataset at the start of each epoch ensures that mini-batches are formed differently across epochs, preventing the network from overfitting to a fixed batch ordering. The loss on a held-out **validation set** is monitored throughout training to assess generalisation — a topic developed in the next session.

5.4. Setting Up for Optimisation

▹ Session 5: Why Feature Scale Matters, Input normalisation, Initialization: The Symmetry Problem, Initialization: Vanishing and Exploding Signals, Xavier and He Initialization

Before running any optimisation algorithm, two practical steps have a large effect on whether training succeeds: normalising the inputs and initialising the weights carefully. Both affect the geometry of the loss landscape and the behaviour of gradients from the very first step.

5.4.1. Input normalisation

The shape of the loss surface as a function of the parameters depends on the scale of the input features. To see why, consider a concrete example close to home. Suppose we want to predict the future income of MAI graduates based on two features: x_1, their average final grade in the programme (ranging from 1 to 4), and x_2, the total number of days they spent in the programme (ranging from 0 to roughly 1500). We fit a linear regression model with MSE loss:

$$\mathcal{L}(\theta_1, \theta_2; \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} \left(\theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - y^{(i)} \right)^2, \tag{5.6}$$

where θ_1 is the weight on the grade feature and θ_2 is the weight on the duration feature. The partial derivatives with respect to the two parameters are:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{2}{N} \sum_i r^{(i)} x_1^{(i)}, \qquad \frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{2}{N} \sum_i r^{(i)} x_2^{(i)}, \tag{5.7}$$

where $r^{(i)} = \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} - y^{(i)}$ is the residual. The gradient with respect to each parameter is scaled by the corresponding feature values. Since x_2 takes values up to 1500 while x_1 ranges only over $[1, 4]$, we have $x_2 \gg x_1$, and therefore $\partial \mathcal{L}/\partial \theta_2 \gg \partial \mathcal{L}/\partial \theta_1$ for the same residual. A tiny change in θ_2 causes a large change in the loss, while a large change in θ_1 barely moves it at all.

The consequence is that the loss contours are highly elongated ellipses, as shown in Figure 5.4 (top): the loss is very sensitive in the θ_2 direction and nearly flat in the θ_1 direction. Gradient descent oscillates violently across the steep direction while making negligible progress along the flat one — a classic symptom of an ill-conditioned optimisation problem.

To see what this means in practice, consider what happens during training without normalisation. The gradient with respect to θ_2 is large, so the update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$ makes a large step in the θ_2 direction. If η is chosen to make reasonable progress along θ_2, the step in the θ_1 direction is tiny — because the gradient there is small. The optimiser effectively ignores θ_1 for many iterations. Conversely, if η is increased to make progress on θ_1, the step in θ_2 overshoots, causing the iterates to bounce back and forth across the valley without descending into it. There is no single learning rate that works well for both parameters simultaneously. In the worst case, training diverges: the overshoot in the θ_2 direction grows with each step, and the loss increases rather than decreasing. In the best case, training is simply very slow — the optimiser creeps along in the flat θ_1 direction while wasting most of its steps oscillating across θ_2. Both pathologies become dramatically worse as the number of features grows and their scales become more varied.

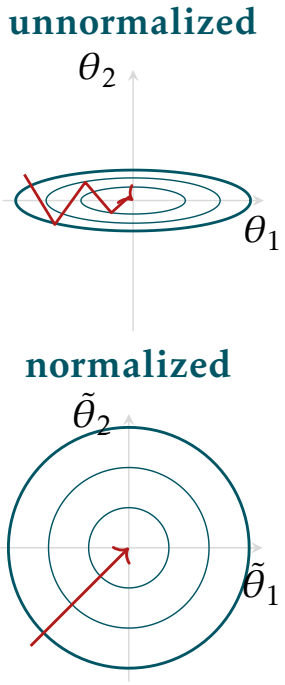

Figure 5.4: Unnormalised (top) vs normalised (bottom) loss contours.

The root cause is not the model or the data, but the mismatch in scales between the two features. A student's grade and the number of days they spent studying are both meaningful predictors of income, but they happen to be measured in very different units. The optimiser has no way to know this — it sees only numbers, and large numbers produce large gradients.

The standard fix is **z-score normalisation**, applied feature-wise before training:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \qquad \mu_j = \frac{1}{N}\sum_{i=1}^{N} x_j^{(i)}, \quad \sigma_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_j^{(i)} - \mu_j)^2}. \tag{5.8}$$

After normalisation, the grade and duration features both have zero mean and unit variance. The optimiser now sees both on equal footing: a unit change in $\tilde{x}_1$ has the same scale as a unit change in $\tilde{x}_2$, and both gradients are of similar magnitude. The loss contours become approximately circular (Figure 5.4, bottom), and gradient descent converges much faster.

A critical implementation detail: μ_j and σ_j must be computed on the **training set only** and then applied identically to validation and test sets. Computing statistics on the full dataset would leak information about the test distribution into preprocessing — a subtle but consequential data leakage error.

5.4.2. Weight Initialisation

The choice of initial weights affects optimisation just as much as the loss landscape geometry. We already saw in Section 6.2.4 and Figure 5.3 that two trajectories starting

from nearby but distinct points can converge to entirely different minima. The initialisation therefore determines not only how fast training proceeds, but which region of the loss landscape is explored and what solution is ultimately found. There are two distinct failure modes to avoid.

The symmetry problem. If all weights are initialised to the same value — for instance, all zeros, or any other fixed constant — then all neurons in a layer combine the same inputs with the same weights, and therefore compute identical pre-activations:

$$z_j^{(l)} = \sum_i \theta_{ji}^{(l)} h_i^{(l-1)} + \theta_{j0}^{(l)} = c \quad \forall\, j. \tag{5.9}$$

Every neuron in the layer produces the same output, and since all outputs are identical, every neuron in the *next* layer also receives the same inputs — propagating the symmetry forward through the entire network.

The problem compounds during the backward pass. Because every neuron in a layer has the same output, the loss is equally sensitive to each of them, and their gradients are identical. This means every neuron receives the same gradient signal and makes the same parameter update. After the update, the weights are still all equal — just shifted by the same amount. The symmetry is never broken, no matter how many gradient steps are taken.

The practical consequence is that the network has the width of a single neuron per layer, regardless of its architecture. All the neurons in a layer learn the same function, and the expressive capacity that width is supposed to provide is completely wasted. Weights must therefore be initialised **randomly** to break this symmetry — ensuring that different neurons start from different points and specialise in different directions during training.

Vanishing and exploding signals. Random initialisation is necessary but not sufficient — the scale of the random weights matters critically. To see why, consider an L-layer network with linear activations and weights drawn independently with $\mathbb{E}[\theta] = 0$ and $\mathrm{Var}[\theta] = \sigma^2$, with n_{in} inputs per layer. The pre-activation at each layer is a weighted sum of n_{in} independent inputs:

$$z_j^{(l)} = \sum_{i=1}^{n_{\text{in}}} \theta_{ji}^{(l)} h_i^{(l-1)}. \tag{5.10}$$

Since the weights and inputs are independent and zero-mean, the variance of this sum is the sum of the variances of the individual terms. Each term $\theta_{ji} h_i$ has variance $\mathrm{Var}[\theta] \cdot \mathrm{Var}[h] = \sigma^2 \cdot \mathrm{Var}[h]$, and there are n_{in} such terms, giving:

$$\mathrm{Var}[z^{(l)}] = n_{\text{in}}\, \sigma^2 \cdot \mathrm{Var}[h^{(l-1)}]. \tag{5.11}$$

With linear activations, $h^{(l)} = z^{(l)}$, so this relation applies recursively. After L layers:

$$\mathrm{Var}[z^{(L)}] = (n_{\text{in}}\, \sigma^2)^L \cdot \mathrm{Var}[x]. \tag{5.12}$$

The factor $n_{\text{in}}\, \sigma^2$ is multiplied once per layer, so the signal variance grows or shrinks geometrically with depth. This has severe practical consequences.

If $n_{\text{in}}\,\sigma^2 < 1$, the variance shrinks at every layer. By the time the signal reaches the output in a deep network, it is effectively zero — activations are all near zero, and the network cannot distinguish between different inputs. More critically, backpropagation propagates gradients *backward* through the same weights, so the gradient variance shrinks by the same factor at each layer. The gradients reaching the early layers are vanishingly small: those layers receive almost no learning signal and their weights barely move during training, regardless of how many gradient steps are taken. The network learns only from its final layers, and its early feature detectors remain random.

If $n_{\text{in}}\,\sigma^2 > 1$, the variance grows at every layer. Activations become very large, and gradients blow up exponentially as they propagate backward. A single gradient step moves the parameters by an enormous amount, completely undoing any progress made in previous steps. In practice this manifests as loss values that spike to infinity or NaN within the first few iterations of training.

Both failure modes become dramatically worse with depth — a factor of 0.9 per layer is barely noticeable in a three-layer network but reduces the gradient to $0.9^{50} \approx 0.005$ in a fifty-layer one. The only stable regime is $n_{\text{in}}\,\sigma^2 = 1$, which keeps the signal variance constant across all layers and gives the target initialisation variance $\sigma^2 = 1/n_{\text{in}}$.

Remark 4. The derivation above assumes linear activations and is exact under independence. With nonlinear activations the argument is approximate, but the qualitative conclusion — that $n_{\text{in}}\,\sigma^2$ controls stability — remains valid.

Xavier and He initialisation. The condition $\sigma^2 = 1/n_{\text{in}}$ ensures variance is preserved in the forward pass. A symmetric argument applied to the backward pass yields the condition $\sigma^2 = 1/n_{\text{out}}$. These two conditions cannot generally be satisfied simultaneously. Glorot and Bengio (2010) proposed taking their harmonic mean:

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}, \qquad \theta_{ji}^{(l)} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}\right]. \tag{5.13}$$

This is **Xavier** (or Glorot) initialisation, designed for symmetric activations such as tanh or sigmoid.

For ReLU activations, approximately half the neurons are zeroed at any given forward pass, which halves the effective variance relative to the linear case. He et al. (2015) showed that this requires compensating with an extra factor of two:

$$\sigma^2 = \frac{2}{n_{\text{in}}}, \qquad \theta_{ji}^{(l)} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right). \tag{5.14}$$

This is **He** (or Kaiming) initialisation.

5.5. Momentum and Adaptive Methods

▸ Session 5: Limitations of Vanilla GD, Momentum, RMSProp, Adam

Even with good initialisation and normalised inputs, vanilla mini-batch gradient descent can be slow or unstable. Three failure modes motivate progressively more sophisticated update rules.

5.5.1. Momentum

Mini-batch gradients are noisy: each batch produces a slightly different gradient estimate, causing the parameter updates to fluctuate. In a narrow valley — where the loss is steep across the valley and flat along it — this noise produces rapid oscillations across the valley while progress along the valley is slow. **Momentum** addresses this by accumulating a velocity vector $\mathbf{m}_t$, defined as an exponential moving average of past gradients:

$$\mathbf{m}_t = \beta\,\mathbf{m}_{t-1} + (1-\beta)\nabla_\theta \hat{\mathcal{L}}_t, \tag{5.15}$$

$$\theta \leftarrow \theta - \eta\,\mathbf{m}_t. \tag{5.16}$$

The coefficient $\beta \in [0,1)$ — typically 0.9 — controls how much history is retained. Gradients in directions that are consistent across many steps accumulate in $\mathbf{m}_t$, amplifying progress along those directions. Gradients in directions that alternate sign (oscillations across a narrow valley) cancel out, damping the oscillations. The net effect is faster, smoother progress along the loss landscape.

This raises an apparent tension with our earlier argument that mini-batch noise is beneficial for landscape exploration. The resolution is that momentum does not remove the stochasticity — it smooths the gradient *direction* across recent batch updates, but each update still uses a different random batch. The randomness that drives landscape exploration comes from sampling different data points each time; that remains intact. What momentum removes is the erratic variation in step direction caused by the high variance of individual batch gradients — the kind of oscillation that makes the optimiser zigzag across a valley without making progress. The two ideas are therefore complementary: stochastic batching provides the exploratory randomness, and momentum channels that into more directed, efficient steps.

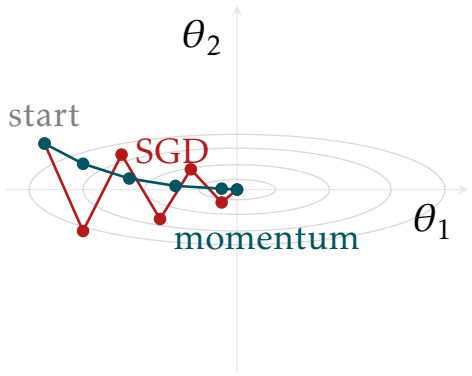

Figure 5.5: SGD oscillates across a narrow valley. Momentum follows a smoother path.

Figure 5.5 contrasts SGD and momentum on an elongated loss landscape. SGD zigzags across the narrow direction; momentum dampens the oscillations and converges more directly.

5.5.2. RMSProp

Momentum smooths the update direction but uses the same effective learning rate for every parameter. In practice, different parameters may have very different gradient magnitudes — some are sensitive (large gradients) and others are nearly flat (small gradients) — and a single η is simultaneously too large for the former and too small for the latter. This problem persists even after input normalisation: normalisation fixes the scale mismatch at the input, but does not control what happens inside the network. As weights evolve during training, different parameters in different layers develop very different gradient magnitudes, and this imbalance changes continuously.

RMSProp addresses this by tracking, for each parameter separately, how large its gradients have been recently. The natural measure is the squared gradient, which

removes the sign so that both positive and negative gradients contribute equally to the magnitude estimate. RMSProp maintains an exponential moving average of these squared gradients:

$$\mathbf{v}_t = \gamma\,\mathbf{v}_{t-1} + (1-\gamma)\left(\nabla_{\boldsymbol{\theta}}\hat{\mathcal{L}}_t\right)^2, \tag{5.17}$$

with $\gamma \approx 0.99$. A parameter with consistently large gradients accumulates a large $\mathbf{v}_t$; one with small gradients accumulates a small $\mathbf{v}_t$. The update divides by $\sqrt{\mathbf{v}_t}$ to normalise the gradient by its recent typical magnitude:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{\mathbf{v}_t + \epsilon}}\nabla_{\boldsymbol{\theta}}\hat{\mathcal{L}}_t, \tag{5.18}$$

with $\epsilon \approx 10^{-8}$ for numerical stability. Parameters with large recent gradients receive a smaller effective step; those with small gradients receive a larger one — automatically equalising the effective learning rate across all dimensions.

The key distinction from momentum is that RMSProp accumulates squared gradients and carries no directional information. Momentum is about smoothing the *direction* of updates; RMSProp is about adapting their *magnitude*. The two are complementary, which motivates combining them.

5.5.3. Adam

Adam (Kingma and J. Ba 2015) (Adaptive Moment Estimation) combines momentum and RMSProp into a single update rule, maintaining both a smoothed gradient direction and a per-parameter magnitude estimate simultaneously:

$$\mathbf{m}_t = \beta_1\,\mathbf{m}_{t-1} + (1-\beta_1)\,\nabla_{\boldsymbol{\theta}}\hat{\mathcal{L}}_t, \tag{5.19}$$

$$\mathbf{v}_t = \beta_2\,\mathbf{v}_{t-1} + (1-\beta_2)\left(\nabla_{\boldsymbol{\theta}}\hat{\mathcal{L}}_t\right)^2. \tag{5.20}$$

Both moving averages are initialised at zero, which biases them toward zero early in training when few gradient estimates have been accumulated. Adam corrects for this by rescaling both estimates:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}. \tag{5.21}$$

When t is small, β^t is close to 1 and the correction is large, inflating the early underestimates back to a reasonable scale. As t grows, $\beta^t \to 0$, the corrections approach 1, and the bias correction quietly switches off. The parameter update is then:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}\,\hat{\mathbf{m}}_t. \tag{5.22}$$

The default hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ work well across a wide range of architectures and tasks, which is why Adam has become the standard default optimiser in deep learning — it requires little tuning and handles the varied gradient scales that arise naturally in deep networks.

5.6. Learning Rate Schedules

▸ Session 5: Why Schedule the Learning Rate?, Learning Rate Schedules

A fixed learning rate is a compromise: a large η enables fast initial progress but overshoots near a minimum; a small η allows fine-grained convergence but is impractically slow early on. **Learning rate schedules** vary η over the course of training to get the best of both regimes. Figure 5.6 illustrates the three schedules discussed below.

Step decay. The simplest schedule reduces η by a multiplicative factor $\gamma < 1$ every k epochs:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/k \rfloor}. \tag{5.23}$$

The staircase pattern visible in Figure 5.6 is characteristic: η is held constant for k steps then drops abruptly. This is simple and interpretable, but the sudden drops can cause instability and the flat regions between drops waste the opportunity for finer progress as the optimiser approaches a minimum.

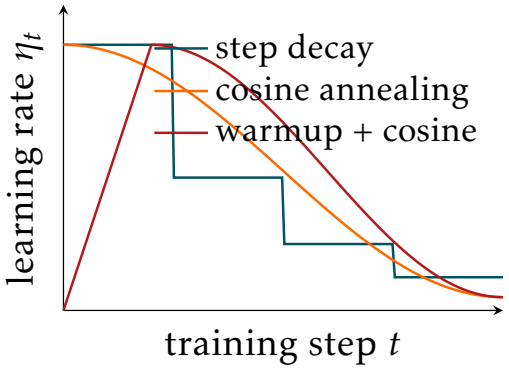

Figure 5.6: Three learning rate schedules.

Cosine annealing. A smooth alternative follows a cosine curve from η_0 down to a minimum $\eta_{\min}$:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_0 - \eta_{\min})\Big(1 + \cos\tfrac{\pi t}{T}\Big), \tag{5.24}$$

where T is the total number of training steps. The decay is fast at first — when the optimiser is still far from a minimum and large steps are useful — and slows gradually as training progresses. This smooth profile avoids the instabilities of step decay and is widely used in modern training recipes.

Linear warmup with cosine decay. Large-scale training — particularly for Transformers — uses a warmup phase in which η is ramped up linearly from zero over the first t_w steps, followed by cosine decay:

$$\eta_t = \begin{cases} \eta_0 \cdot t/t_w & t \le t_w \\ \eta_{\min} + \tfrac{1}{2}(\eta_0 - \eta_{\min})\Big(1 + \cos\tfrac{\pi(t-t_w)}{T-t_w}\Big) & t > t_w. \end{cases} \tag{5.25}$$

The warmup is motivated by the state of the network at the very start of training: parameters are randomly initialised and far from any good solution, so gradients can be large and unreliable. Starting with a small η and ramping it up gradually prevents these early noisy gradients from causing large destabilising updates before the optimiser has settled into a reasonable region of the loss landscape.

6. Generalization & Regularization

▹ Session 7

These notes accompany the seventh lecture. Sessions 1–6 built the full machinery for training deep networks: forward pass, backpropagation, optimisation, and the PyTorch API. Throughout, we minimised a training loss and measured progress by how low it went. This session steps back and asks a more fundamental question: is minimising training loss the right objective? The answer is no - the true goal is performance on new, unseen data from the population. We develop the concept of generalisation, examine why deep networks generalise surprisingly well despite being massively overparameterised, and then study the main techniques for ensuring generalisation in practice.

6.1. Generalisation

▹ Session 7: Where We Are, True Goal, Train / Validation / Test Split, Overfitting and Underfitting, What To Do

We begin by formalising what a good model actually means and establishing the tools for measuring it.

6.1.1. True Goal and Generalisation Gap

Throughout this course we have trained models by minimising the empirical loss on a training dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta; \mathcal{D}) = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(\mathbf{x}^{(i)}), y^{(i)}). \tag{6.1}$$

But what we actually want is a model that performs well on **new, unseen examples drawn from the same population**. The true objective is the **population loss**:

$$\mathcal{L}_{\text{pop}}(\theta) = \mathbb{E}_{(\mathbf{x},y)\sim p_{\text{data}}}\Big[\ell(f_\theta(\mathbf{x}), y)\Big], \tag{6.2}$$

where p_{data} is the true data-generating distribution. This is an **expectation** - the average loss over all possible examples from the population. On some examples the model will do better than average, on others worse; this reflects natural variation in the data and is unavoidable. Optimising the average is the right objective. The training set $\mathcal{D}$ is a finite sample from p_{data}, and minimising $\mathcal{L}(\theta; \mathcal{D})$ is only a proxy for minimising $\mathcal{L}_{\text{pop}}(\theta)$.

The **generalisation gap** quantifies how well the proxy serves the true objective:

$$\text{generalisation gap} = \mathcal{L}_{\text{pop}}(\theta) - \mathcal{L}_{\text{train}}(\theta) \geq 0. \tag{6.3}$$

A model with a small generalisation gap has learned something genuinely useful about the data-generating process. A model with a large gap has overfit - it has memorised properties of the training set that do not hold for the population.

6.1.2. Train, Validation, and Test Split

In practice, a dataset is divided into three non-overlapping subsets with distinct roles:

Training set - used to fit model parameters by minimising $\mathcal{L}(\theta; \mathcal{D}_{\text{train}})$. The model sees this data repeatedly during training.

Validation set - used to evaluate the model during development: selecting architecture, tuning hyperparameters, and deciding when to stop training. The model does not train on validation data, but validation performance influences decisions about the model. It is therefore touched many times during development.

Test set - used for final, honest evaluation of the deployed model. It is a held-out sample from p_{data} used to estimate $\mathcal{L}_{\text{pop}}$ - a **measurement tool, not the true objective**. Repeatedly evaluating on the test set and adjusting the model is a form of **overfitting the test set**: the test loss becomes an optimistic estimate of true population performance. This is why the test set must be touched only once, at the very end of development, and must not influence any modelling decision.

Remark 5. The key requirement for validation and test sets is that they are **representative of the population** - they should reflect the full variation in p_{data}. A more complex population requires larger sets to estimate performance reliably. At the same time, every example allocated to validation or test is unavailable for training,

so there is a tradeoff. Typical splits of 80/10/10 or 70/15/15 are reasonable starting points, but the right split depends on dataset size and population complexity.

6.1.3. Overfitting and Underfitting

Two failure modes arise from the training-population gap:

Underfitting occurs when the model is too simple to capture the structure in the data. Both training loss and test loss are high, and the generalisation gap is small - the model fails on training data and test data alike. Remedies include increasing model capacity, training longer, or tuning the learning rate and optimiser.

Overfitting occurs when the model is too complex and memorises training data, including its noise. Training loss is low but test loss is high, producing a large generalisation gap. The model has learned patterns that are specific to the training set and do not generalise. Remedies include regularisation (discussed in section 6.3), obtaining more data, or reducing model capacity.

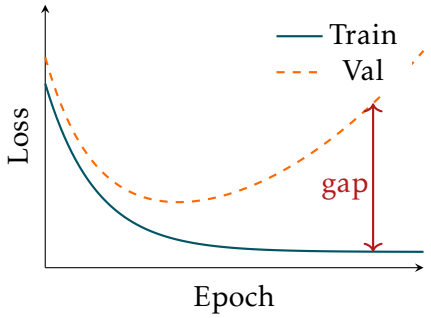

Figure 6.1: Training and validation loss over epochs. Train loss decreases monotonically; validation loss initially decreases then increases as the model begins to overfit. The gap between the two curves is the generalisation gap.

The primary diagnostic tool is monitoring the generalisation gap during training - figure 6.1 shows an example plot of training loss and validation loss together as a function of epochs. When validation loss stops improving and begins to rise while training loss continues to fall, overfitting is occurring.

6.2. Generalisation Paradox

▹ Session 7: Bias-Variance Tradeoff, Limits of Classical View, Many Good Minima, Flat vs Sharp Minima, Double Descent

Classical statistical learning theory makes a clear prediction: models with more parameters than training examples should overfit catastrophically. Deep networks violate this prediction routinely - they are massively overparameterised yet generalise well. This section examines why, starting from the classical bias-variance framework and then developing the modern understanding of why deep networks are different.

6.2.1. Bias-Variance Tradeoff

Classical statistical learning theory decomposes the expected test error into two competing terms. The **bias** measures how wrong the model is on average across all possible training sets - a high-bias model is too simple to capture the true data-generating process. The **variance** measures how sensitive the model is to the specific training set used - a high-variance model fits each training set very well but produces different predictions for different training sets, so it fails on new data.

The classical prescription is to choose model complexity to balance the two: simple models have high bias and low variance (underfitting), complex models have low bias and high variance (overfitting), and the optimal model sits somewhere in between. Adding more parameters reduces bias but increases variance - so classical theory predicts that overparameterised models should overfit badly.

6.2.2. Limits of the Classical View

Deep networks have huge numbers of parameters - far more than needed to memorise the training data. They routinely drive training loss close to zero, nearly memorising the training set. Classical bias-variance theory predicts this should lead to catastrophic overfitting. Yet in practice, deep networks generalise well. This is the **generalisation paradox** of deep learning, and it is not fully resolved by classical theory. Understanding why deep networks generalise requires looking beyond the classical framework.

6.2.3. Many Good Minima

One part of the explanation is that overparameterised networks have a vast number of local minima, and most of them have very similar loss values. Networks trained from different random initialisations converge to different minima yet achieve similar generalisation performance. This means the choice of which minimum gradient descent finds is less critical than classical theory suggests - there are simply many equivalent solutions, and the optimiser finds one of them.

A theoretical underpinning for this observation comes from the connection to statistical physics. Choromanska et al. (2015) showed, using a correspondence with spin-glass models, that in deep networks the local minima with high loss become exponentially rare as network size grows - almost all local minima cluster near the global minimum. The more practically relevant obstacle is not local minima at all but **saddle points** - critical points where the loss curves upward in some directions and downward in others. Dauphin et al. (2014) argued that in high-dimensional spaces, saddle points vastly outnumber local minima, and that escaping saddle points is the main challenge for gradient-based optimisation in deep networks. The noise introduced by mini-batch SGD helps here too: stochastic gradient estimates provide perturbations that can push iterates away from saddle regions where the exact gradient would stall.

6.2.4. Flat vs Sharp Minima

Not all minima generalise equally well. A **sharp minimum** is one where the loss changes rapidly as parameters move away from the minimum - the basin is narrow and deep. A **flat minimum** is one where the loss changes slowly - the basin is wide and shallow. Figure 6.2 illustrates the difference.

The connection between flatness and generalisation was first formalised by Hochreiter and Schmidhuber (1997a), who argued using minimum description length that flat minima correspond to simpler, more compressible models and therefore generalise better. The intuition is the following: when we move from the training distribution to the test distribution, the loss surface shifts slightly. A sharp minimum that is optimal for the training distribution may be far from optimal for the test distribution - a small shift in the surface is enough to push the parameters out of the narrow basin, causing a large increase in

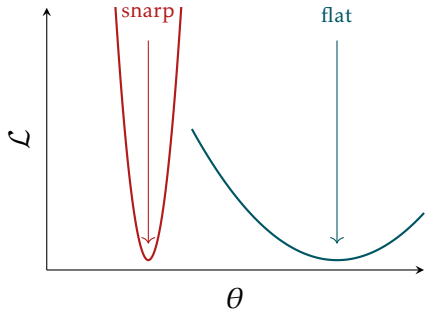

Figure 6.2: Sharp and flat minima. The sharp minimum (carnelian) has high curvature - a small shift in parameters causes a large increase in loss. The flat minimum (petrol) has low curvature - loss remains low over a wide region of parameter space.

loss. A flat minimum is more robust: even if the surface shifts, the parameters remain in a region of low loss.

The practical importance of this observation was demonstrated by Keskar et al. (2017), who showed empirically that large-batch SGD consistently converges to sharp minima that generalise poorly, while small-batch SGD finds flat minima that generalise well. This provides a direct explanation for the well-known generalisation gap between large-batch and small-batch training, and confirms that mini-batch SGD acts as an **implicit regulariser** - it biases the optimiser toward solutions that generalise better, without any explicit change to the loss function.

6.2.5. Double Descent

The classical bias-variance curve predicts a single U-shaped relationship between model complexity and test error - one sweet spot of optimal complexity. Belkin et al. (2019) showed that this picture is incomplete: the **double descent** phenomenon reveals that beyond the interpolation threshold, test error decreases again as complexity grows further.

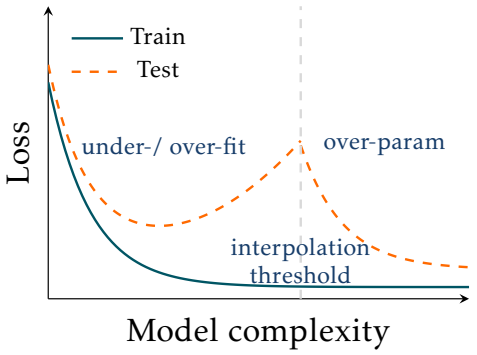

Figure 6.3: Double descent. The classical U-shaped bias-variance curve (left) predicts a single optimal complexity. Beyond the interpolation threshold, test loss decreases again as model complexity grows further.

As model complexity grows, test error first follows the classical U-shape. At the **interpolation threshold** - the point where the model has just enough capacity to fit the training data exactly - test error peaks. This is the worst operating point: the model is maximally rigid, forced to commit every degree of freedom to fitting training points exactly, with no slack for smooth interpolation. Beyond this threshold, as complexity continues to grow, test error decreases again. Nakkiran et al. (2020) demonstrated that this phenomenon occurs in deep neural networks specifically, appearing not only as a function of model size but also as a function of training epochs and dataset size.

The intuition for why overparameterisation helps is that excess capacity allows smoother, more natural interpolation between training points. When a model has just enough parameters to fit the training data, every degree of freedom is fully committed - the interpolant is maximally constrained and can produce sharp, jagged behaviour between training points. When the model is overparameterised, there are infinitely many solutions that fit the data exactly, and gradient descent from a small random initialisation selects among them. Bartlett, Montanari, and Rakhlin (2020) showed for linear models that gradient descent finds the **minimum-norm solution** - the interpolant that fits the training data while keeping the parameter vector as small as possible. Minimum-norm solutions are smooth by construction: large oscillations between training points require large parameter magnitudes, so minimising the norm suppresses them.

This connects back to the **implicit regularisation of mini-batch SGD** discussed in section 6.2.4. The noise in gradient estimates not only favours flat minima over sharp ones, but also biases the optimiser toward low-norm solutions. Both effects work in

the same direction: the optimisation procedure itself acts as a regulariser, steering training toward solutions that generalise well - without any explicit penalty term in the loss. Deep networks operate in this overparameterised regime, to the right of the interpolation threshold, and benefit from both effects.

6.3. Regularisation Techniques

▹ Session 7: Weight Decay, Dropout – Training, Dropout – Evaluation, Dropout – Ensemble Interpretation, Data Augmentation, Early Stopping

SGD provides implicit regularisation for free. We now turn to explicit techniques that can be added on top to further control the generalisation gap.

6.3.1. Weight Decay

Large weights indicate that the model relies heavily on specific features - small changes in input produce large changes in output, increasing sensitivity to the training set and reducing generalisation. The classical fix is to penalise large weights directly by adding a regularisation term to the loss:

$$\tilde{\mathcal{L}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\boldsymbol{\theta}\|^2. \tag{6.4}$$

This is **L2 regularisation**, also called **weight decay** - the gradient of the penalty term, $\lambda\boldsymbol{\theta}$, shrinks all weights toward zero at each update step, distributing reliance more evenly across features. The hyperparameter $\lambda > 0$ controls the strength and is tuned on the validation set.

In PyTorch, weight decay is available via the `weight_decay` parameter within optimisers. Weight decay is a useful starting point but it is less central in deep learning than in classical ML. Mini-batch SGD already provides implicit regularisation by biasing toward low-norm solutions, reducing the marginal benefit of an explicit penalty. Moreover when using Adam, standard L2 regularisation interacts undesirably with the adaptive learning rates (Loshchilov and Hutter 2019).

6.3.2. Dropout

Dropout (Srivastava et al. 2014) is one of the most powerful and widely used regularisation techniques in deep learning. It randomly zeros each activation during training with probability p, called the **dropout rate**. Each forward pass effectively uses a different subnetwork - a random subset of the neurons, as illustrated in Figure 6.4.

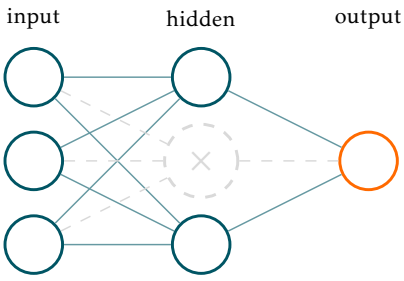

Figure 6.4: Dropout during training. One neuron (grey, crossed) is randomly zeroed. Each forward pass uses a different subnetwork.

Dropout prevents neurons from **co-adapting**: no neuron can rely on specific other neurons always being present, because those neurons may be absent in the next forward pass. Each neuron is therefore forced to be independently useful - it must learn a feature that is valuable on its own, not one that only makes sense in combination with specific other neurons. On the other hand, it also prevents **over-specialisation**: neurons are discouraged from learning highly specific, narrow features that are only useful in one context. The result is a network that learns more distributed,

robust representations - each neuron contributes meaningfully on its own, and the representations generalise better to unseen data.

Train and evaluation modes. During training, only a fraction $(1-p)$ of neurons are active on average - the network learns with reduced capacity. During evaluation the behaviour changes in two ways. First, all neurons are active - no masking is applied. Second, the outputs must be rescaled to account for the difference in the number of active neurons between training and evaluation. Without correction, the full network at test time would produce larger activations than the network was trained to expect, changing the effective scale of every layer.

The standard correction is to multiply activations by $(1-p)$ at test time. An equivalent and more computationally convenient formulation is **inverted dropout**: scale active activations *up* by $\frac{1}{1-p}$ during training instead, so that no correction is needed at evaluation. This is what PyTorch implements - `nn.Dropout(p)` scales active activations by $\frac{1}{1-p}$ during training and passes activations through unchanged during evaluation. The net effect is the same; the advantage is that the evaluation forward pass requires no extra operations.

Typical values of p range from 0.1 to 0.5, tuned on the validation set. Dropout works best when applied to **fully connected layers**. In other architectures - convolutional networks (CNNs), recurrent networks (RNNs), transformers - the standard formulation is less effective or requires adaptation. Dropout also interacts with batch normalisation in a non-trivial way; this is addressed in section 6.4.1.

Ensemble interpretation. A deeper understanding of why dropout works comes from the ensemble perspective. A network with n neurons and dropout rate p implicitly defines 2^n different subnetworks - one for each possible dropout mask. During training, each mini-batch trains a different subnetwork, but all subnetworks share the same weights. This is an extremely parameter-efficient form of ensemble training: instead of training 2^n separate models, a single set of weights is shared across all of them.

At test time, using the full network with rescaled activations approximates averaging the predictions of all 2^n subnetworks simultaneously. Ensemble averaging is a classical and well-understood technique for reducing variance: individual models may overfit in different directions, but their average tends to cancel out idiosyncratic errors and generalise better. Dropout achieves this ensemble effect at the cost of training and deploying a single model - a remarkable result given the exponential number of subnetworks implicitly represented.

6.3.3. `model.train()` and `model.eval()`

During training we want certain layers to behave stochastically - dropout should randomly zero activations, producing a different subnetwork for each forward pass. During evaluation and inference we want deterministic, stable behaviour - dropout should be disabled and all neurons should be active. These are two distinct operational regimes that require different behaviour from the same layers. Other layers also have dual behaviour - most notably batch normalisation, which we will encounter in section 6.4.1.

PyTorch implements this through a **model mode**: every `nn.Module` instance is either in *training mode* or *evaluation mode*. The mode is set by calling `model.train()` or `model.eval()` on your instantiated model, where `model` is whatever name you gave

it. Both calls propagate recursively to all submodules, so a single call on the top-level model is sufficient regardless of how many layers it contains. Layers that respond to the mode switch include `nn.Dropout` and, as we will see later, `nn.BatchNorm`; all other standard layers - `nn.Linear`, `nn.ReLU`, and so on - are unaffected.

The correct pattern in a training loop is:

```
model = MyNetwork(...)        # your instantiated model

# training loop
model.train()
for X_batch, y_batch in train_loader:
    optimizer.zero_grad()
    loss = loss_fn(model(X_batch), y_batch)
    loss.backward()
    optimizer.step()

# validation loop
model.eval()
with torch.no_grad():
    for X_batch, y_batch in val_loader:
        loss = loss_fn(model(X_batch), y_batch)
```

Remark 6. Forgetting to call `model.eval()` before validation or inference is a silent bug - no error is raised, but predictions are wrong because dropout is still active and batch normalisation uses incorrect statistics. Always call `model.eval()` before validation and `model.train()` before resuming training.

6.3.4. Data Augmentation

Data augmentation **creates new training examples** by applying transformations to existing ones from the training set. The fundamental requirement is **invariance**: the correct prediction must be invariant to the transformation, which means the transformation must be **label-preserving**. These are two ways of saying the same thing - if rotating an image of a cat still produces an image of a cat, the model should be invariant to rotation, and rotation is therefore a valid label-preserving transformation.

This constraint gives augmentation two complementary benefits. First, it **increases the effective size of the training set** - more diverse training examples reduce overfitting regardless of their content. Second, and more fundamentally, it **explicitly teaches the model the invariances that matter for the problem**: a model trained on cats in many orientations learns that orientation does not affect the label; a model trained on patient measurements with small added noise learns that minor measurement variation does not affect the diagnosis.

For image data, common augmentations include random horizontal flips, rotations, crops, and colour jitter; for tabular data, Gaussian noise on continuous features and random feature masking are typical choices. Though these are reasonable starting points, they are not universally valid.

The choice of transformations requires **domain knowledge** - only the practitioner knows which invariances are valid for their specific problem. Every augmentation is an

implicit claim about the data-generating process: it asserts that transformed examples are plausible draws from p_{data} with the same label. A horizontal flip is valid for natural image classification - a cat seen from the left is the same cat seen from the right. The same flip applied to digit recognition is wrong - a 6 rotated 180° becomes a 9. Aggressive colour changes are harmless for classifying animals but damaging for medical imaging, where colour carries diagnostic information. **Augmentation pipelines from papers and tutorials must never be transferred blindly to a new problem.** Every augmentation must be justified by asking: is this transformation label-preserving for *my* problem and *my* data?

In practice, augmentation is applied **on the fly during data loading**: at each training epoch, transformations are sampled randomly and applied to training examples as they are loaded into a mini-batch. The training set itself is never modified - the model sees a different random version of each example at every epoch, continuously expanding the effective variety of training data.

Note that invariance can also be built directly into the architecture rather than learned from augmented data - certain architectures are invariant to specific transformations by construction. This will become relevant when we study convolutional networks.

6.3.5. Early Stopping

Training longer always reduces training loss - the optimiser will keep finding ways to fit the training data more precisely. But beyond a certain point this comes at the cost of generalisation: the model begins to memorise training-specific patterns that do not hold for the population, and the generalisation gap grows. The validation loss, which tracks performance on held-out data, reveals this: it initially decreases alongside training loss, but eventually levels off and begins to rise as overfitting sets in, as illustrated in Figure 6.5.

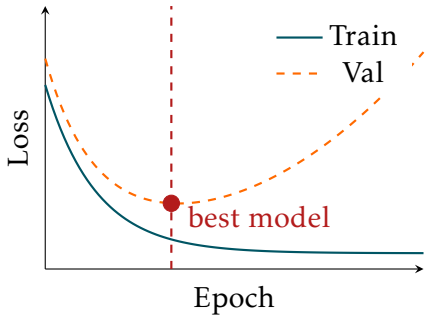

Figure 6.5: Early stopping. Training loss decreases monotonically; validation loss eventually increases. The best model is saved at the validation minimum and deployed rather than the final model.

The natural response is to stop training at the point where the model generalises best - the minimum of the validation loss curve - rather than training until convergence. This is **early stopping**. It is a form of regularisation because it constrains the effective training time: a model trained for fewer steps has had less opportunity to memorise the training data, limiting its effective capacity without changing the model architecture or the loss function.

In practice, the best model encountered during training is saved at each epoch and the final deployed model is this saved checkpoint, not the model at the end of training. Since validation loss is noisy and may fluctuate temporarily before resuming its descent, training is not stopped at the first sign of non-improvement. Instead, a **patience** parameter k specifies how many consecutive epochs without improvement to tolerate before stopping. All the ingredients for early stopping are already present in the training loop from session 6 - it simply adds a stopping criterion and model checkpointing.

6.4. Training Stability

▹ Session 7: Training Stability, Internal Covariate Shift, Batch Normalisation, Vanishing and Exploding Gradients, Dying ReLU, Activation Functions

Regularisation solves the problem of overfitting - it allows us to use large, overparameterised networks without the model memorising the training data. But deeper networks introduce a separate class of problems that have nothing to do with overfitting: **training instabilities**. These are problems that make training impossible or unreliable regardless of whether the model would overfit. More layers amplify small issues, and three instabilities are particularly important: internal covariate shift, vanishing and exploding gradients, and the dying ReLU problem.

6.4.1. Internal Covariate Shift and Batch Normalisation

In section 5.4.1 we saw that normalising the inputs to the network stabilises training: unnormalised features with very different scales produce an ill-conditioned loss surface and make gradient descent behave poorly. The same problem occurs at every layer of a deep network, not just the first one - and it is aggravated by the fact that training is a dynamic process.

Consider a layer l receiving activations from layer $l-1$ as inputs. The distribution of outputs of layer l depends on two things: the data and the parameters of all preceding layers. After a gradient update, both change simultaneously. The parameters of layer l and all preceding layers update, changing the distribution of outputs each layer produces affecting all subsequent layers down the road. This effect compounds with depth: a layer deep in the network receives inputs that have been transformed through many layers, each of which updated its parameters, so the distribution it sees can shift dramatically between update steps. The non-linearity amplifies this further - a shift in the pre-activation distribution can produce a very different post-activation distribution, potentially pushing activations into saturating regions where gradients vanish, or dramatically increasing the variance of the activations. This phenomenon of changing distributions within layers is called **internal covariate shift** (Ioffe and Szegedy 2015).

The consequences are the same as for unnormalised inputs: slower convergence, need for lower learning rates, and high sensitivity to initialisation. The solution follows the same logic too - if normalising the inputs to the network stabilises the first layer, normalising the activations at every layer stabilises all of them. This is exactly what **batch normalisation** does: it explicitly normalises the activations at each layer to have zero mean and unit variance within each mini-batch, then applies learnable affine transformations:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \qquad \tilde{h}_j^{(i)} = \gamma_j \hat{h}_j^{(i)} + \beta_j, \qquad \mu_j = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} h_j^{(i)}, \quad \sigma_j^2 = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(h_j^{(i)} - \mu_j)^2 \tag{6.5}$$

where μ_j and σ_j^2 are the mean and variance of feature j computed across all examples $i \in \mathcal{B}$ in the current mini-batch, ϵ is a small constant for numerical stability, and γ_j and β_j are learnable per-feature scale and shift parameters that allow the normalised activation to be rescaled and shifted. By keeping the input distribution to each layer approximately stable, batch normalisation allows higher learning rates, reduces sensitivity to initialisation, and makes training of very deep networks practical.

During training, the normalisation uses statistics computed from the current mini-batch - mean and variance are different for each batch, injecting a small amount of

noise into the normalisation. This is analogous to the noise in mini-batch SGD and has a **mild implicit regularisation effect**, though this is not the primary motivation for using batch normalisation.

During evaluation, mini-batch statistics are replaced by **running statistics** - exponential moving averages of $\mu_{\mathcal{B}}$ and $\sigma^2_{\mathcal{B}}$ accumulated during training. This gives deterministic, stable behaviour at test time: the normalisation no longer depends on which other examples happen to be in the same batch. This is why batch normalisation is one of the layers affected by `model.train()` and `model.eval()` - forgetting to call `model.eval()` before inference means the network normalises using noisy mini-batch statistics rather than the stable running statistics, producing wrong and irreproducible predictions.

In PyTorch, `nn.BatchNorm1d` is used for fully connected networks and `nn.BatchNorm2d` for convolutional networks - covered in a later session. Batch norm is typically inserted after the linear transformation and before the activation function.

Batchnorm and Dropout. Using batch norm and dropout together requires care. The fundamental problem is a **variance shift** (Li et al. 2019): during training with dropout active, some activations are zeroed, reducing the variance of the distribution entering batch norm. The running statistics accumulated during training therefore reflect this reduced variance. At test time, dropout is disabled and all activations are present - the variance is higher, but the running statistics used for normalisation were computed under the lower-variance training distribution. This mismatch produces systematically wrong normalisation at test time and degrades performance. In practice, three approaches are used:

- **Batch norm only** - sufficient in many cases, particularly in convolutional networks where batch norm provides strong regularisation on its own
- **Dropout only** - effective in MLPs with no interaction problems
- **Both** - possible if dropout is placed *after* batch norm, so that running statistics are computed on the full activations; results may still be suboptimal and require careful tuning

An alternative that avoids these complications entirely is **layer normalisation** (J. L. Ba, Kiros, and G. E. Hinton 2016). Rather than normalising across the batch dimension per feature, layer norm normalises across the feature dimension per example:

$$\hat{h}_j^{(i)} = \frac{h_j^{(i)} - \mu^{(i)}}{\sqrt{\sigma^{2(i)} + \epsilon}}, \qquad \tilde{h}_j^{(i)} = \gamma_j \hat{h}_j^{(i)} + \beta_j \qquad \mu^{(i)} = \frac{1}{D}\sum_{j=1}^{D} h_j^{(i)}, \quad \sigma^{2(i)} = \frac{1}{D}\sum_{j=1}^{D}(h_j^{(i)} - \mu^{(i)})^2 \tag{6.6}$$

where $\mu^{(i)}$ and $\sigma^{2(i)}$ are the mean and variance computed across all D features of example i, ϵ is a small constant for numerical stability, and γ_j and β_j are learnable per-feature scale and shift parameters. Because statistics are computed per example independently, there are no running statistics to accumulate and no train/eval difference - layer norm behaves identically in both modes. This means layer norm and dropout combine cleanly. In MLPs, the typical order is: `nn.Linear` → `nn.LayerNorm` → activation → `nn.Dropout`. Transformers use a related but slightly different pattern involving residual connections - this will be covered when we study sequence models.

Batch norm and layer norm are two points in a larger design space of normalisation techniques. **Group norm** normalises across groups of channels per example and is useful when batch sizes are small. **RMS norm** is a simplified layer norm without mean

subtraction, used in modern large language models such as LLaMA. The field has not settled on a single winner - the right choice depends on the architecture, batch size, and task.

6.4.2. Vanishing and Exploding Gradients

The second major training instability arises during backpropagation. The gradient of the loss with respect to the parameters of the first layer involves a product of Jacobians across all layers:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \cdot \frac{\partial \mathbf{h}^{(L)}}{\partial \mathbf{h}^{(L-1)}} \cdots \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \boldsymbol{\theta}^{(1)}}. \tag{6.7}$$

This product of L Jacobian matrices compounds as it travels backwards from the loss to the earliest layers. If the singular values of the Jacobians are consistently less than 1, the product shrinks exponentially with depth - **vanishing gradients**. The earliest layers receive gradients close to zero, their parameters barely update, and the deep network effectively behaves as a shallow one. If the singular values are consistently greater than 1, the product grows exponentially - **exploding gradients**. The earliest layers receive enormous gradients, a single update step moves the parameters by a huge amount, and training diverges.

For standard MLPs and CNNs, **Xavier and He initialisation** discussion in section 5.4.2 and **batch normalisation** of section 6.4.1 largely solve the vanishing and exploding gradient problem in practice. Initialisation keeps the Jacobian singular values close to 1 at the start of training; batch normalisation stabilises the forward pass distributions throughout training, which indirectly stabilises the backward pass. Gradient instability is therefore rarely a practical concern for these architectures.

The situation is more severe in recurrent networks and transformers, where gradients must propagate through long sequences of dependencies. Here, initialisation and normalisation alone are not sufficient and dedicated solutions are needed. **Gradient clipping** rescales the gradient if its norm exceeds a threshold τ before the update step, preventing any single step from being catastrophically large. In PyTorch: `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`, called between `loss.backward()` and `optimizer.step()`. **Spectral normalisation** constrains the spectral norm of weight matrices, bounding the Jacobian singular values by construction. Both are standard tools in recurrent and attention-based architectures, which we will study in later sessions.

6.4.3. Dying ReLU and Activation Functions

The third instability is the **dying ReLU** problem. The ReLU activation function has a zero gradient for all negative pre-activations:

$$\frac{\partial \mathrm{ReLU}(z)}{\partial z} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0. \end{cases} \tag{6.8}$$

This sounds alarming: ReLU has zero gradient for half the input space. In a deep network, the vanishing gradient problem would seem to be inevitable.

In practice, the dying ReLU problem is less severe than it first appears. A neuron dies only if its pre-activation is negative for *all* training examples simultaneously - in a diverse dataset, pre-activations vary across examples and the average

gradient over a mini-batch is nonzero. He initialisation further reduces the risk by keeping pre-activations in a healthy range at the start of training. Batch normalisation helps too: by keeping the pre-activation distribution centred at zero throughout training, it prevents the drift toward predominantly negative values that would increase the risk of neurons dying. It also guarantees that approximately half the pre-activations remain positive at all times - and for sufficiently wide networks, this is always enough active neurons to represent the necessary functions. ReLU therefore remains the recommended baseline: simple, computationally cheap, and well-protected by the standard training recipe of He initialisation and batch normalisation.

When dying ReLU does become a problem, the fix is to use an activation function that has a nonzero gradient for negative inputs. **Leaky ReLU** introduces a small negative slope α:

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \alpha \ll 1. \tag{6.9}$$

ELU (Exponential Linear Unit) uses a smooth exponential curve for negative inputs, avoiding the hard zero entirely. More recent activations - **GELU**, **Swish**, and **Mish** - are smooth, differentiable everywhere, and are used in modern architectures such as transformers. There is no universal winner; ReLU and LeakyReLU cover most practical needs for the feedforward networks considered in this course.

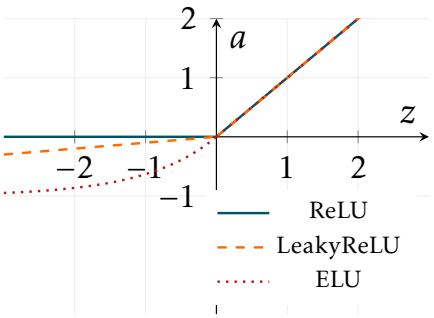

Figure 6.6: ReLU, LeakyReLU ($\alpha = 0.1$), and ELU. LeakyReLU introduces a small negative slope; ELU uses a smooth exponential curve. Both maintain a nonzero gradient for negative pre-activations.

6.5. Embracing Randomness

▹ Session 7: Embracing Randomness, Summary

A recurring observation throughout this lecture is that randomness appears at every stage of deep learning training - not as a nuisance to be eliminated but as a deliberate feature that contributes to generalisation. Table 1 summarises the main sources of randomness and their effects.

Source	Where	Effect
Weight initialisation	start of training	which minimum is found
Data shuffling	each epoch	different batches, prevents order memorisation
Mini-batch SGD	every update step	implicit regularisation, flat minima
Dropout	forward pass	ensemble effect, prevents co-adaptation
Data augmentation	data loading	invariance, population coverage

Table 1: Sources of randomness in deep learning training and their effects on generalisation.

What is striking is that none of these are accidents. Each reflects a deliberate design choice that the field has converged on precisely because it improves generalisation. Weight initialisation is random to break symmetry and explore different regions of the loss landscape. Shuffling is random to prevent the model from exploiting the order of examples. Mini-batch SGD is stochastic by design - full-batch gradient descent is

available but rarely used, because the noise from mini-batching implicitly regularises. Dropout injects noise into the forward pass deliberately. Data augmentation applies random transformations by choice.

The deeper insight is that the right question is not how to make training more deterministic, but how to design the right kinds of randomness. This theme extends well beyond the techniques covered in this lecture. More advanced methods in representation learning, generative modelling, and self-supervised learning return to randomness again and again - as a source of supervision, as a form of regularisation, or as the fundamental mechanism by which a model learns to represent uncertainty. Embracing randomness, rather than treating it as an obstacle, is one of the defining characteristics of modern deep learning.

Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.

References

Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). "Layer Normalization". In: *arXiv preprint arXiv:1607.06450*.

Bartlett, Peter L., Andrea Montanari, and Alexander Rakhlin (2020). "Benign Overfitting in Linear Regression". In: *Proceedings of the National Academy of Sciences*. Vol. 117. 48, pp. 30063–30070.

Belkin, Mikhail et al. (2019). "Reconciling Modern Machine-Learning Practice and the Classical Bias-Variance Trade-off". In: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.

Bengio, Yoshua, Olivier Delalleau, and Nicolas Le Roux (2004). "The Curse of Highly Variable Functions for Local Kernel Machines". In: *Advances in Neural Information Processing Systems*.

Choromanska, Anna et al. (2015). "The Loss Surfaces of Multilayer Networks". In: *Proceedings of the 18th International Conference on Artificial Intelligence and Statistics*.

Cybenko, George (1989). "Approximation by Superpositions of a Sigmoidal Function". In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314. DOI: 10.1007/BF02551274.

Dauphin, Yann et al. (2014). "Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization". In: *Advances in Neural Information Processing Systems*.

Deng, Jia et al. (2009). "ImageNet: A Large-Scale Hierarchical Image Database". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Fukushima, Kunihiko (1980). "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position". In: *Biological Cybernetics* 36, pp. 193–202.

Glorot, Xavier and Yoshua Bengio (2010). "Understanding the Difficulty of Training Deep Feedforward Neural Networks". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*. Ed. by Yee Whye Teh and Mike Titterington. Vol. 9. Proceedings of Machine Learning Research. PMLR, pp. 249–256.

Glorot, Xavier, Antoine Bordes, and Yoshua Bengio (2011). "Deep Sparse Rectifier Neural Networks". In: *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 315–323.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Available at https://www.deeplearningbook.org. MIT Press.

He, Kaiming et al. (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034.

Hinton, Geoffrey E., Simon Osindero, and Yee-Whye Teh (2006). "A Fast Learning Algorithm for Deep Belief Nets". In: *Neural Computation* 18.7, pp. 1527–1554.

Hochreiter, Sepp and Jürgen Schmidhuber (1997a). "Flat Minima". In: *Neural Computation* 9.1, pp. 1–42.

– (1997b). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.

Hopfield, John J. (1982). "Neural Networks and Physical Systems with Emergent Collective Computational Abilities". In: *Proceedings of the National Academy of Sciences* 79.8, pp. 2554–2558.

Hornik, Kurt (1991). "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2, pp. 251–257. DOI: 10.1016/0893-6080(91)90009-T.

Ioffe, Sergey and Christian Szegedy (2015). "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *Proceedings of the 32nd International Conference on Machine Learning*, pp. 448–456.

Jumper, John, Richard Evans, Alexander Pritzel, et al. (2021). "Highly Accurate Protein Structure Prediction with AlphaFold". In: *Nature* 596, pp. 583–589.

Keskar, Nitish Shirish et al. (2017). "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima". In: *International Conference on Learning Representations*.

Kingma, Diederik P. and Jimmy Ba (2015). "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton (2012). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep Learning". In: *Nature* 521, pp. 436–444.

LeCun, Yann, Léon Bottou, et al. (1998). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Li, Xiang et al. (2019). "Understanding the Disharmony between Dropout and Batch Normalization by Variance Shift". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2682–2690.

Lighthill, James (1973). *Artificial Intelligence: A General Survey*. Tech. rep. Available at https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf. Science Research Council.

Loshchilov, Ilya and Frank Hutter (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations*.

Ministry of International Trade and Industry (1982). *Outline of Research and Development Plans for Fifth Generation Computer Systems*. Tech. rep. Institute for New Generation Computer Technology.

Minsky, Marvin and Seymour Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Nair, Vinod and Geoffrey E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the International Conference on Machine Learning*.

Nakkiran, Preetum et al. (2020). "Deep Double Descent: Where Bigger Models and More Data Hurt". In: *International Conference on Learning Representations*.

Raina, Rajat, Anand Madhavan, and Andrew Y. Ng (2009). "Large-Scale Deep Unsupervised Learning Using Graphics Processors". In: *Proceedings of the International Conference on Machine Learning*.

Rosenblatt, Frank (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6, pp. 386–408.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams (1986). "Learning Representations by Back-propagating Errors". In: *Nature* 323, pp. 533–536.

Silver, David, Aja Huang, Chris J. Maddison, et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529, pp. 484–489.

Srivastava, Nitish et al. (2014). "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *Journal of Machine Learning Research* 15.1, pp. 1929–1958.

Vapnik, Vladimir (1995). *The Nature of Statistical Learning Theory*. Springer.
Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30.
Werbos, Paul (1974). "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences". PhD thesis. Harvard University.
Widrow, Bernard and Marcian E. Hoff (1960). "Adaptive Switching Circuits". In: *IRE WESCON Convention Record*. Vol. 4, pp. 96–104.